

Python 基礎

目次

- Module 1. Python技術及開發環境介紹
- Module 2. 資料型態與變數
- Module 3. 控制結構 if...else、for、while
- Module 4. 字串、串列、元組、字典、集合
- Module 5. 函式
- Module 6. 物件導向程式設計
- Module 7. 例外處理
- Module 8. 檔案處理
- Module 9. 模組
- Module 10. 裝飾器

Module 1.

Python技術及開發環境介紹

認識 Python 應用

- Python 是一種跨平台 (Cross-platform)、開放原始碼 (Open Source)、擁有大量社群 (Communities)、物件導向 (Object-Oriented Language) 的程式語言，由 Guido van Rossum (Python 最初設計者兼程式架構師) 在 1989 年底發明。
- Python 的語法簡潔、明確，可以整合許多工具來開發各式各樣的功能與服務，例如網站建置、資料分析與視覺化、機器學習、深度學習、視窗表單設計、自動化、遊戲開發、資料庫操作等，都是 Python 的強項。
- Python 目前主流是 3.x 版本，過去的 2.x 版本已經不再被官方維護，強烈建議大家安裝 3.x 版來進行程式開發與學習。

程式的執行方式

- 直譯式語言 (Interpreted Language)

直譯式語言在執行程式碼時，不會事先將整個程式碼編譯成機器語言。而是透過直譯器 (Interpreter) 逐行讀取和執行。這意味著每次執行程式時，程式碼都會被重新翻譯一次。

- 編譯式語言 (Compiled Language)

編譯式語言需要透過編譯器 (Compiler) 將程式碼一次性翻譯成機器語言的可執行檔案，再執行這個可執行檔。編譯過程發生在執行之前，程式碼編譯完成後，便不再需要原始碼或編譯器來執行程式。

直譯式 vs 編譯式 語言差異

直譯式語言 (以 Python 為例)

原始程式碼 (.py)



直譯器逐行翻譯



直接執行

開發快、除錯容易

VS

編譯式語言 (以 Go 為例)

原始程式碼 (.go)



編譯器進行翻譯



產出可執行檔 (.exe)



執行

執行速度快

直譯式 vs 編譯式 執行的差異

直譯式 (Interpreted)

程式在執行時，逐行由直譯器翻譯並立即執行。



Python

特點

- ✓ 不需要事先產生執行檔
- ✓ 跨平台（只要有直譯器）
- ✓ 開發速度快，除錯容易
- ✓ 執行速度較慢

執行流程（以 Python 為例）



原始程式碼
(.py)



Python 直譯器
(Interpreter)



執行結果

範例：hello.py

```
print("Hello, Python!")  
x = 3 + 4  
print("x = ", x)
```



執行指令

python hello.py

編譯式 (Compiled)

程式在執行前，先經由編譯器翻譯成機器碼（執行檔），再執行該執行檔。



Golang

特點

- ✓ 需要先編譯產生執行檔
- ✓ 執行速度快
- ✓ 跨平台需重新編譯
- ✓ 錯誤在編譯階段就能發現

執行流程（以 Go 為例）



原始程式碼
(.go)



Go 編譯器
(Compiler)



執行檔
(hello.exe)



執行結果

範例：hello.go

```
package main  
import "fmt"  
func main() {  
    fmt.Println("Hello, Go!")  
}
```



編譯指令

go build -o hello hello.go



執行指令

./hello

比較項目	直譯式 (Python)	編譯式 (Golang)
執行流程	原始碼 → 直譯器（逐行翻譯並執行） → 結果	原始碼 → 編譯器（產生執行檔） → 執行檔 → 結果
執行速度	較慢	較快
執行檔產生	不需要	需要（編譯後產生執行檔）
跨平台性	高（有直譯器即可）	較低（需在各平台重新編譯）
除錯方式	執行時錯誤才會出現	編譯時期即可發現部分錯誤
開發效率	較快，適合快速開發	較慢（需編譯），但適合大型專案
代表語言	Python, Ruby, JavaScript, PHP...	Go, C, C++, Rust, Java...

建置Python開發環境

- Python 的環境建置來源，通常分為
 - 官方網站 (<https://www.python.org/>)
 - Anaconda (<https://www.anaconda.com/>)
 - Docker (https://hub.docker.com/_/python)



在不同時期，某些套件與Python版本存在相容性的問題

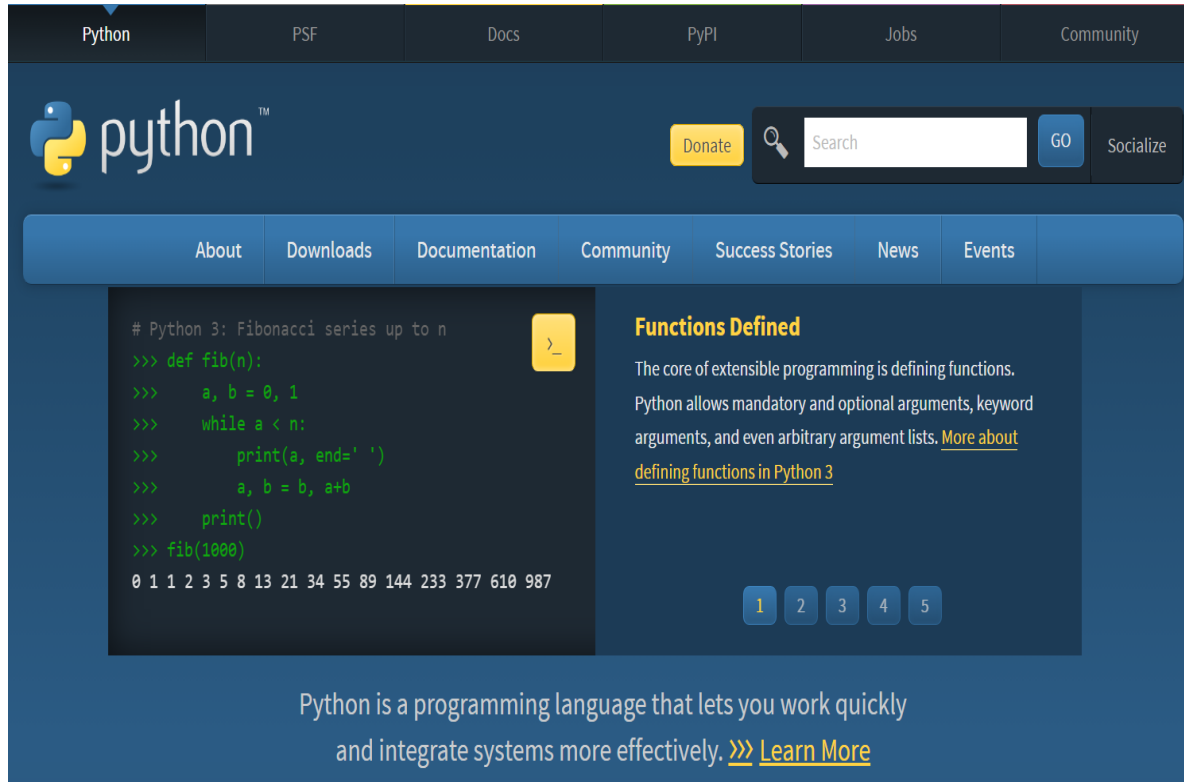
在安裝或使用某些特定套件時，務必要留意Python版本是否相容

套件管理工具 pip

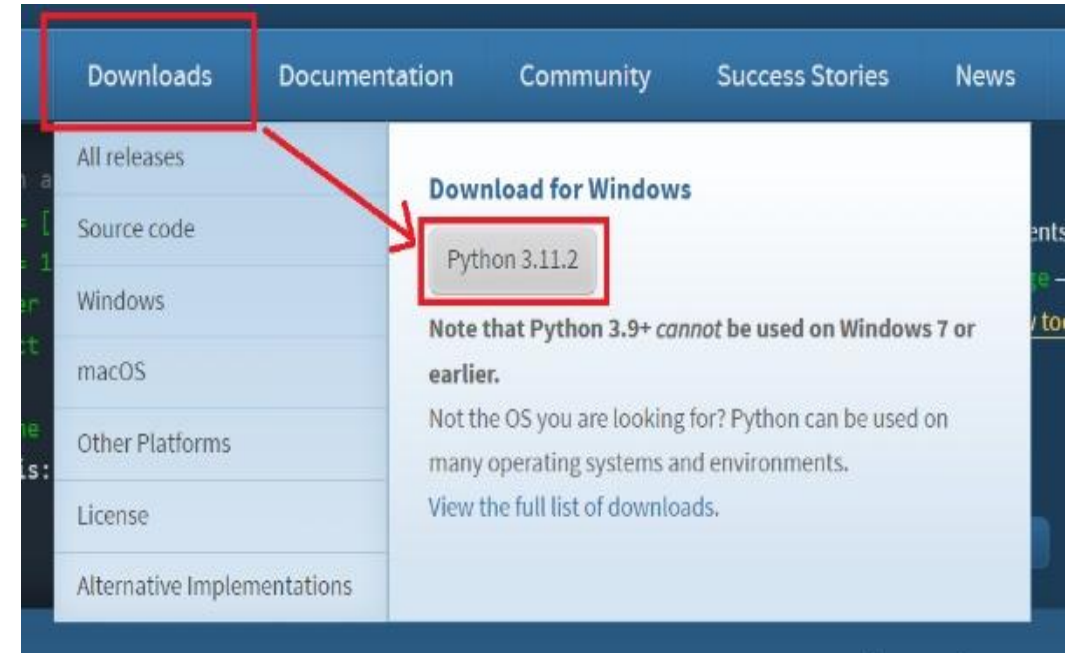
- pip 是 Python 的套件管理工具，可以用來安裝、更新和管理從 Python Package Index (PyPI) 以及其他第三方套件存儲庫中找到的 Python 套件。
- 安裝套件
 - `pip install <套件名稱>`
- 列出所有已安裝的套件
 - `pip list`
- 刪除套件
 - `pip uninstall <套件名稱>`

安裝 Python 單一環境

- 官方網站 (<https://www.python.org/>)



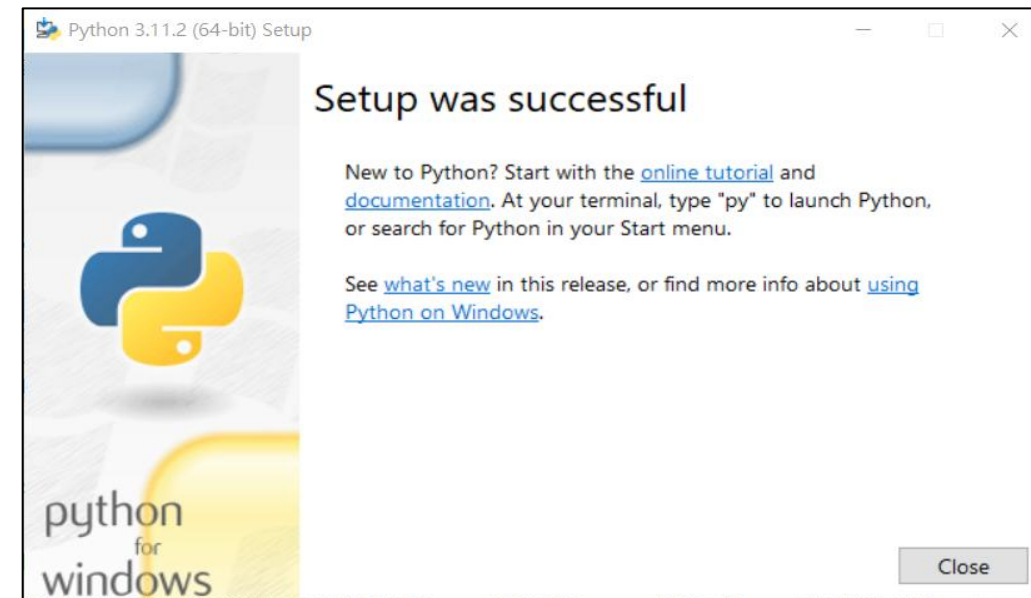
圖：python.org 首頁



圖：游標移至 Download，
看到 Python3.xx 下載按鈕，
直接點它下載安裝程式



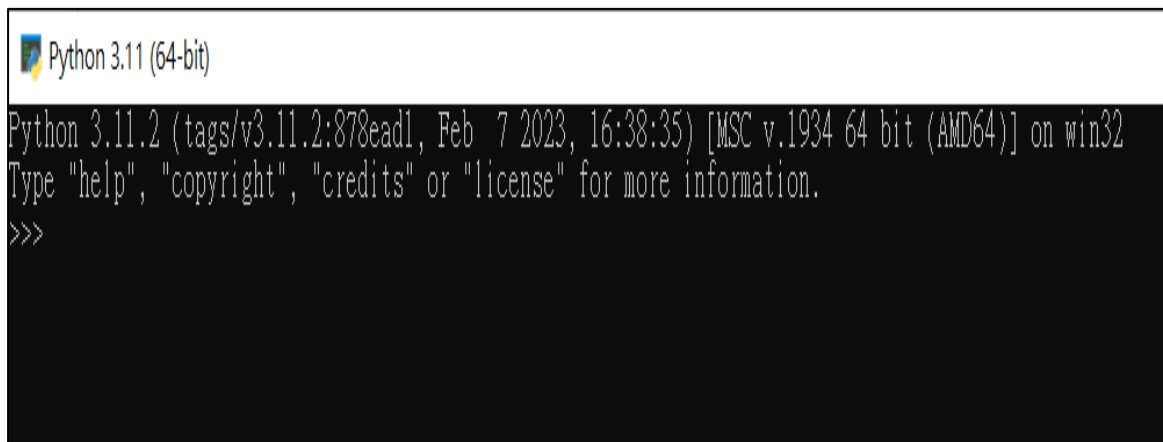
圖：打勾「Add python.exe to PATH」
按下上方的Install Now
(若有安全性提示，可以按下同意)



圖：安裝成功畫面



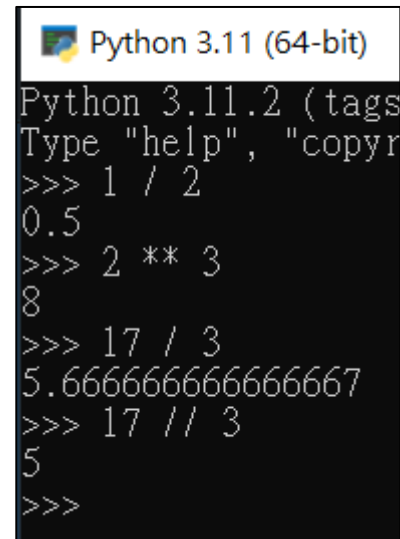
圖：到 Windows 選單，尋找最近新增，選擇「Python 3.xx (64-bit)」



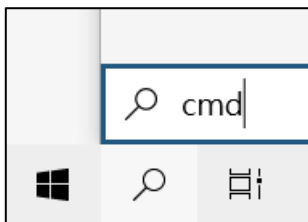
圖：Python 指令操作介面

範例

```
>>> 1 / 2 (輸入完，按下 Enter)
0.5
>>> 2 ** 3 (輸入完，按下 Enter)
8
>>> 17 / 3 (輸入完，按下 Enter)
5.666666666666667
>>> 17 // 3 (輸入完，按下 Enter)
5
```



圖：執行結果。
可輸入 quit() 或 exit()
再按 Enter 離開



圖：搜尋 cmd

按下 Enter，進入命令提示字元

```
命令提示字元 - python
Microsoft Windows [版本 10.0.19045.2604]
(c) Microsoft Corporation. 著作權所有，並保留一切權利。
C:\Users\Owner>python
Python 3.11.2 (tags/v3.11.2:878ead1, Feb 7 2023, 16:38:35) [MSC v.1934 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license" for more information.
>>>
```

圖：在命令提示字元輸入「python」後，按下 Enter，可使用 Python

```
C:\Users\Owner>python -V
Python 3.11.2
C:\Users\Owner>
```

圖：可以在命令提示字元輸入「python -V」，按下 Enter，得知當前 Python 的版本



註：

若是安裝過程中，沒有打勾
「python.exe to PATH」，
無法在命令提示字元中使用
python 指令

如何撰寫 Python 程式

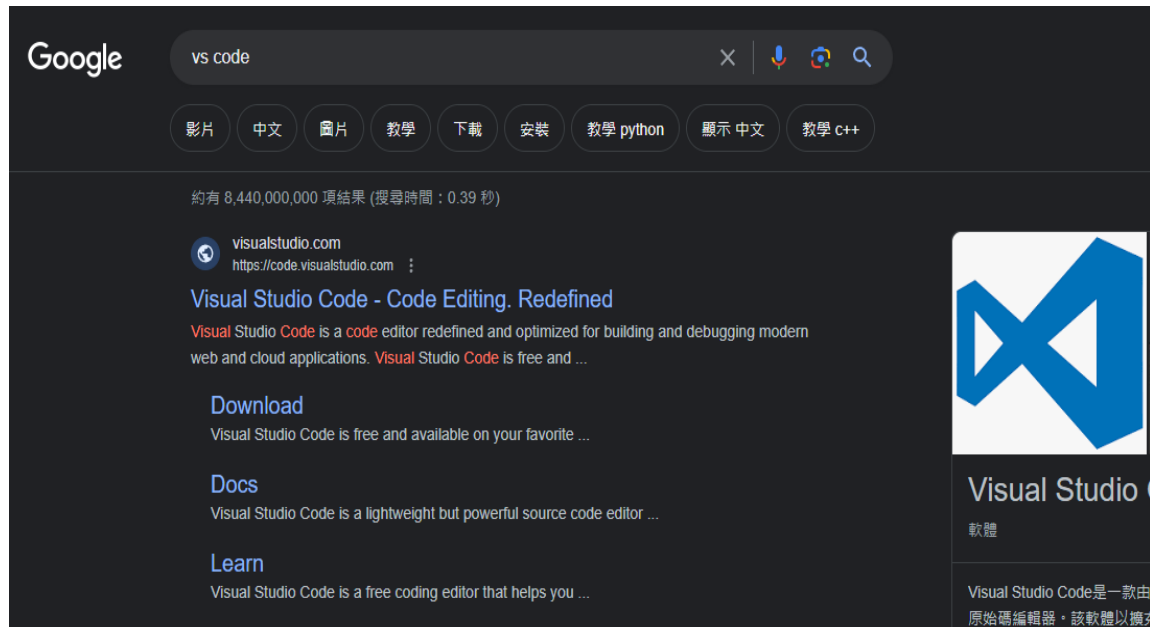
- 使用 IDE

- 整合式開發環境 (Integrated Development Environment, IDE) 可以協助開發者方便使用類似記事本的程式碼輸入介面，側欄又有類似檔案總管的管理功能。常見的 Python IDE 有：

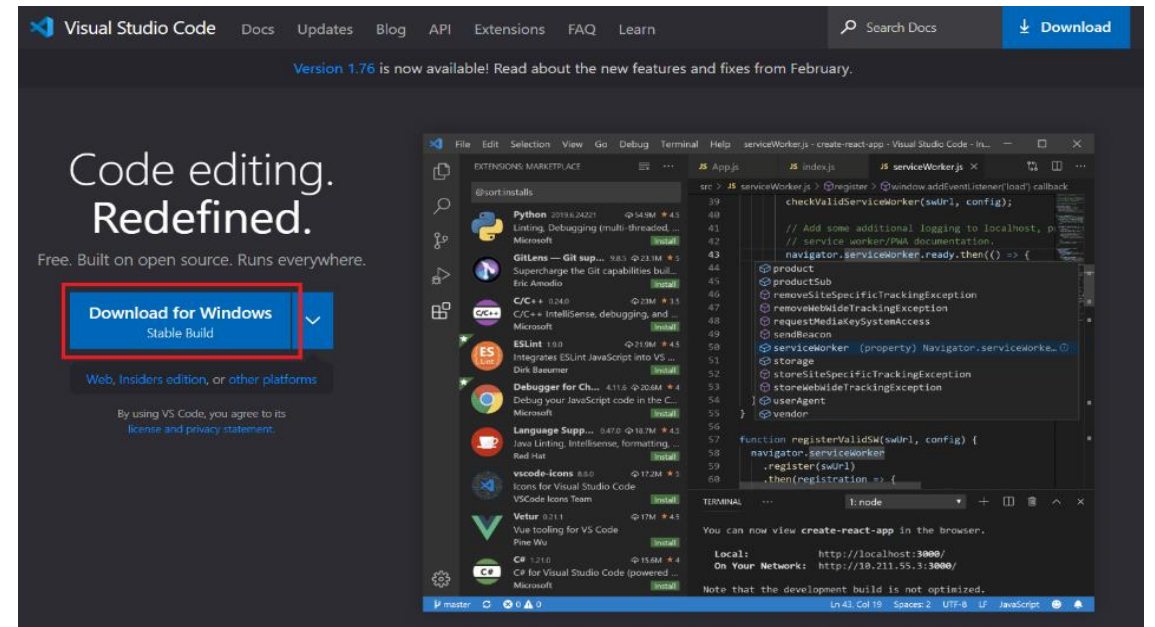
1. Visual Studio Code
2. Antigravity
3. Cursor
4. Pycharm
5. Colab
6. Vim
7. Spyder
8. Eclipse with PyDev
9. Thonny
10. Wing

在課程中，我們選用「Visual Studio Code」（簡稱為 VS code）來作為課程使用的 IDE。VS code 是微軟開發的跨平台（Cross-platform）程式開發工具，除了開發程式之外，還可以使用豐富的擴充功能（Extensions）來提升開發的效率。

VS Code 安裝



圖：在 google 搜尋「vs code」，
進入 vs code 首頁



圖：網站會偵測使用者的作業系統，依需求選擇後，
按下 Download for XXX

Thanks for downloading VS Code for Windows!

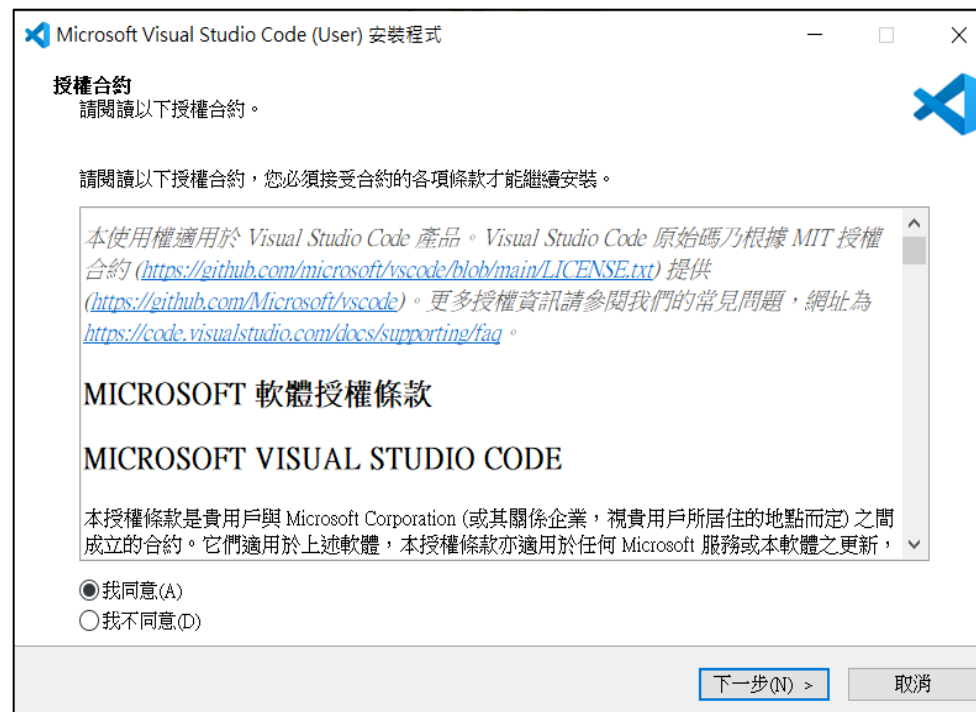
Download not starting? Try this [direct download link](#).

Please take a few seconds and help us improve ... [click to take survey](#).

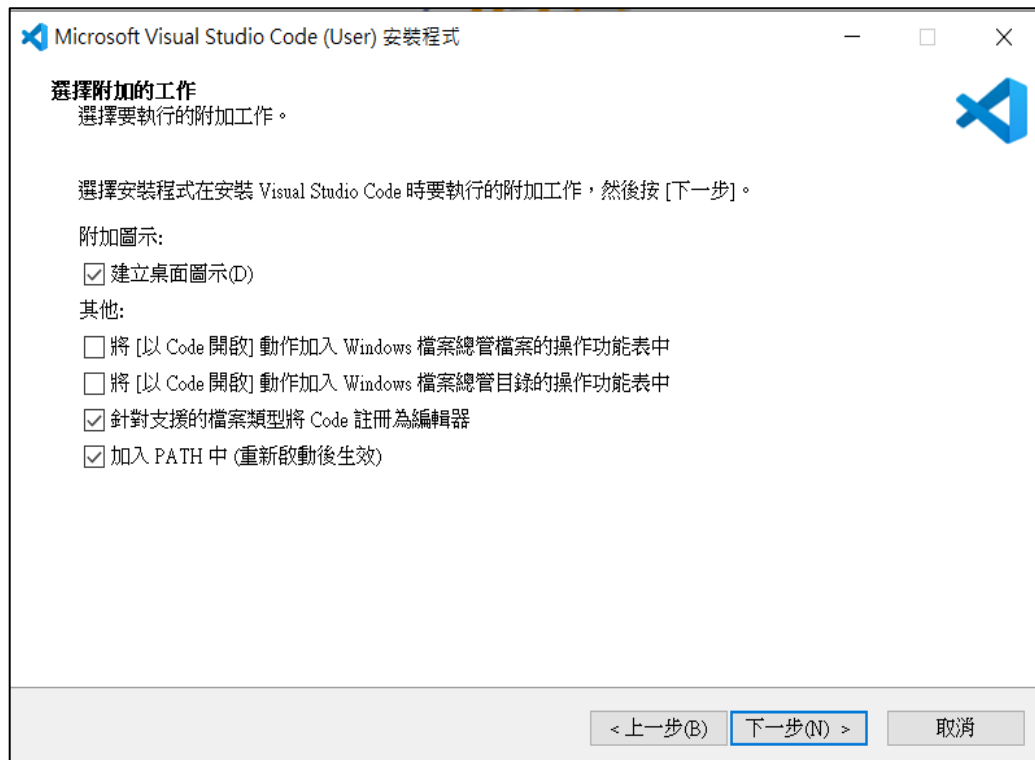
Getting Started

Visual Studio Code is a lightweight but powerful source code editor which runs on your desktop and is available for Windows, macOS and Linux. It comes with built-in support for JavaScript, TypeScript and Node.js and has a rich ecosystem of extensions for other languages and runtimes (such as C++, C#, Java, Python, PHP, Go, .NET). Begin your journey with VS Code with these [introductory videos](#).

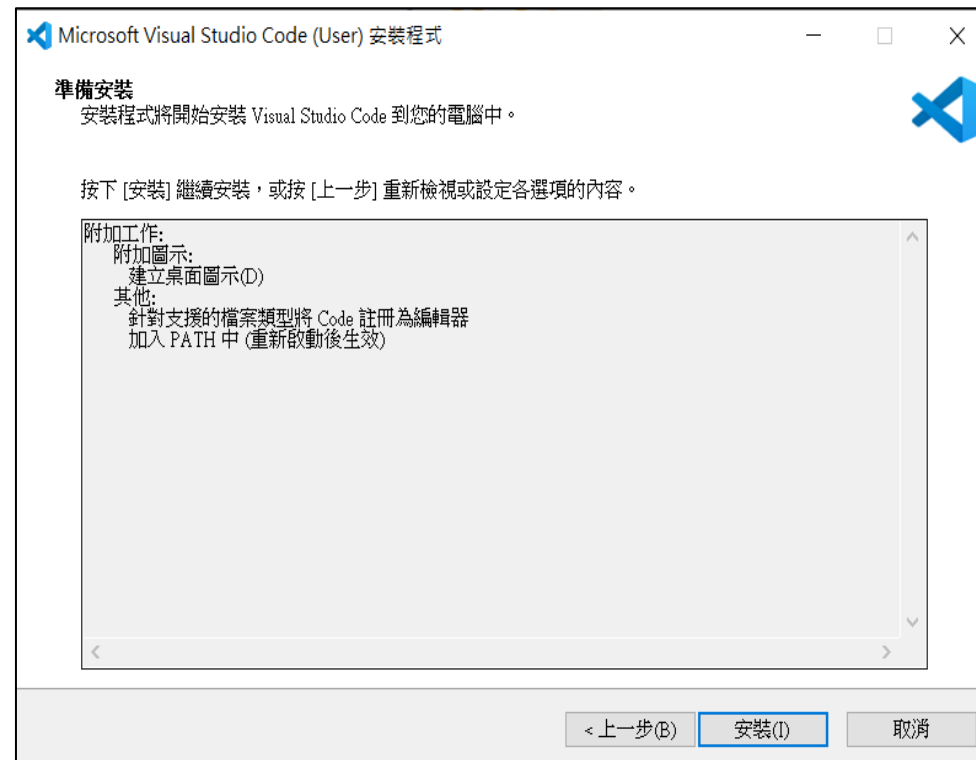
圖：看到這個畫面，安裝檔案會自動下載，
沒有下載的話，
可以按下「direct download link」



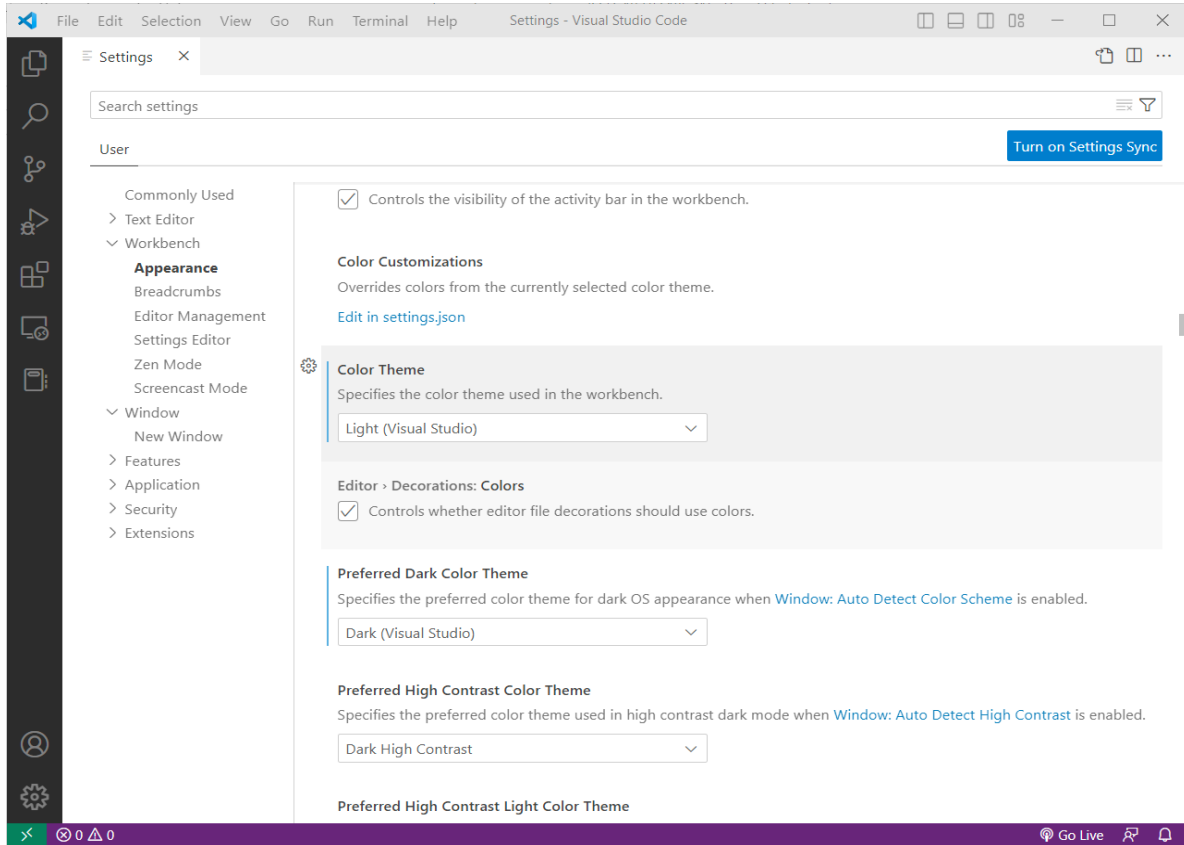
圖：勾選「我同意」，按「下一步」



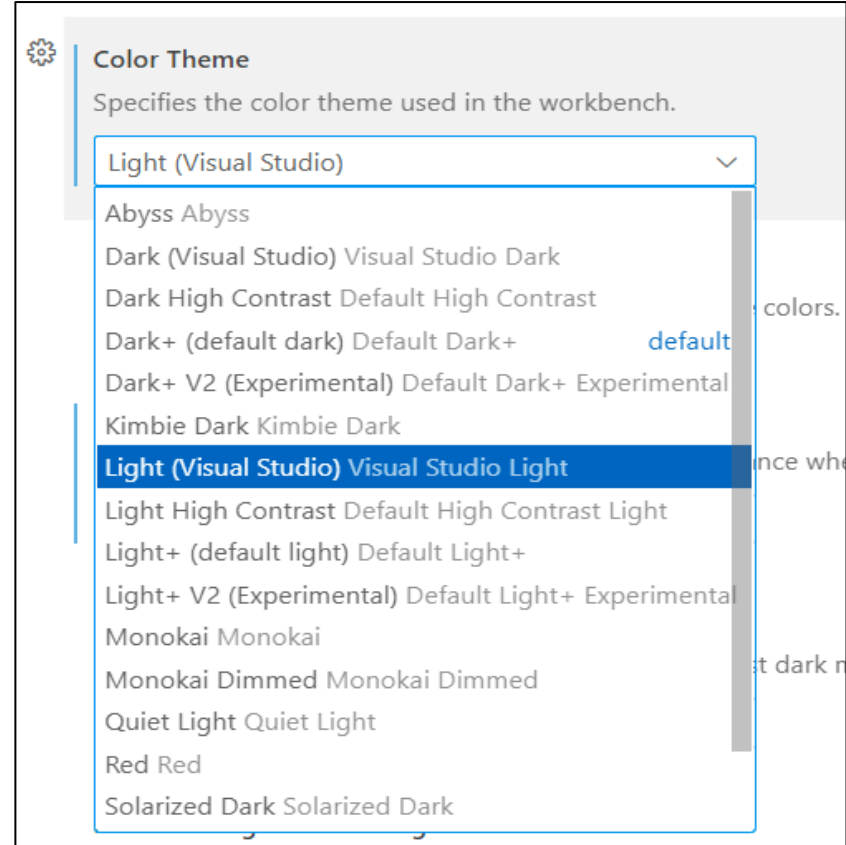
圖：勾選「建立桌面圖示」，按「下一步」



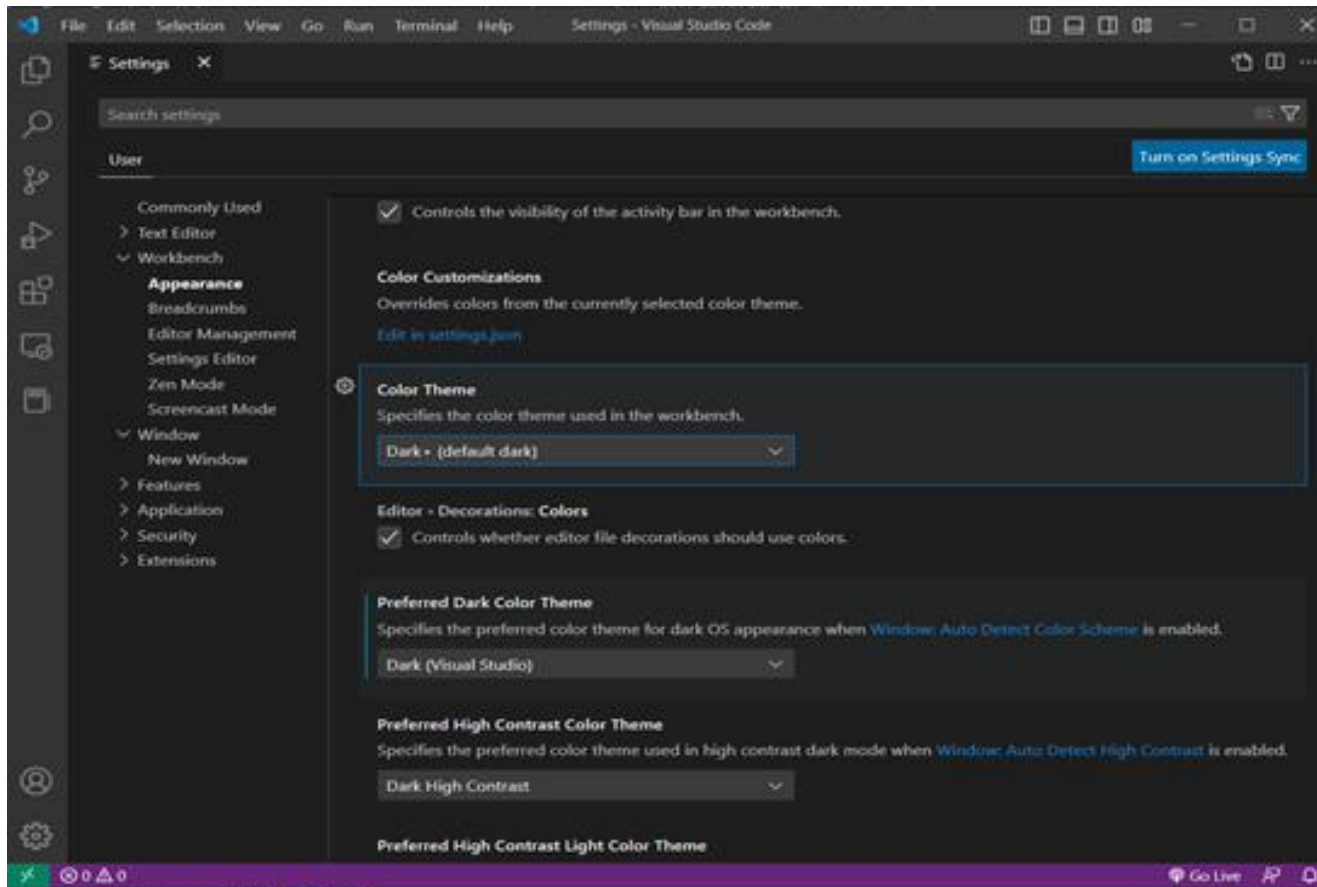
圖：按下「安裝」



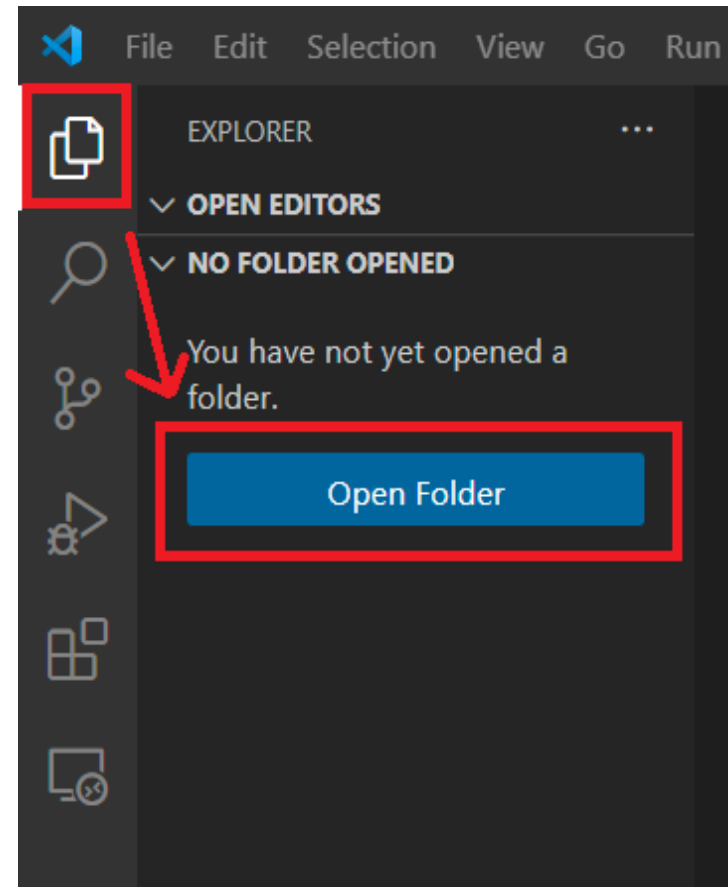
圖：如果想要換佈景顏色，
按下「Ctrl + ,」來開啟設定



圖：在 Color Theme 下方的選單中，
選擇偏好的佈景，例如選擇「Dark+」

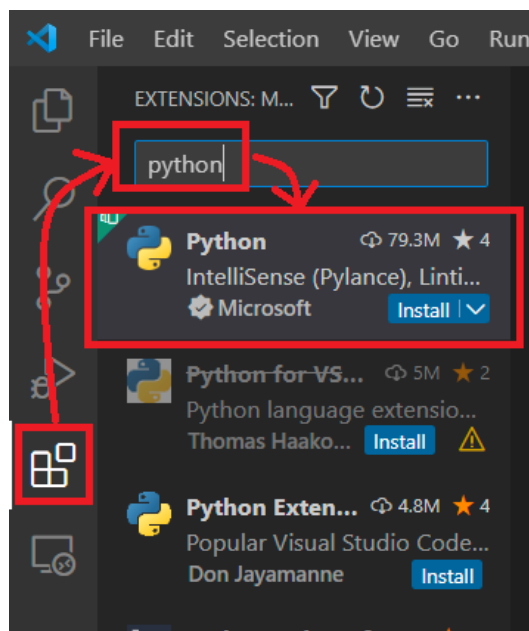


圖：選擇「Dark+」，可以立即套用選擇的佈景，
可以把左上方的頁籤 Settings 關掉（透過按下
Settings 右邊的 x 圖示）



圖：按下左上角檔案總管（Explorer），
再按下開啟資料夾（Open Folder），

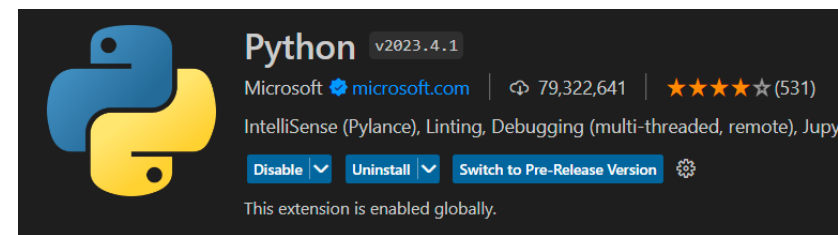
讓專案資料夾使用 Conda 環境



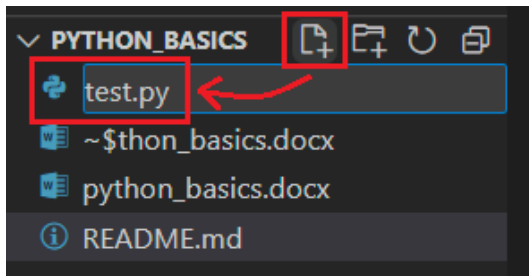
圖：按下擴充功能（Extensions），
在上方輸入「python」，
點選「Python」



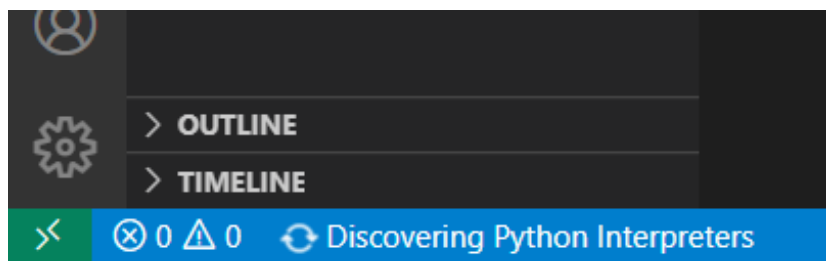
圖：按下 Install



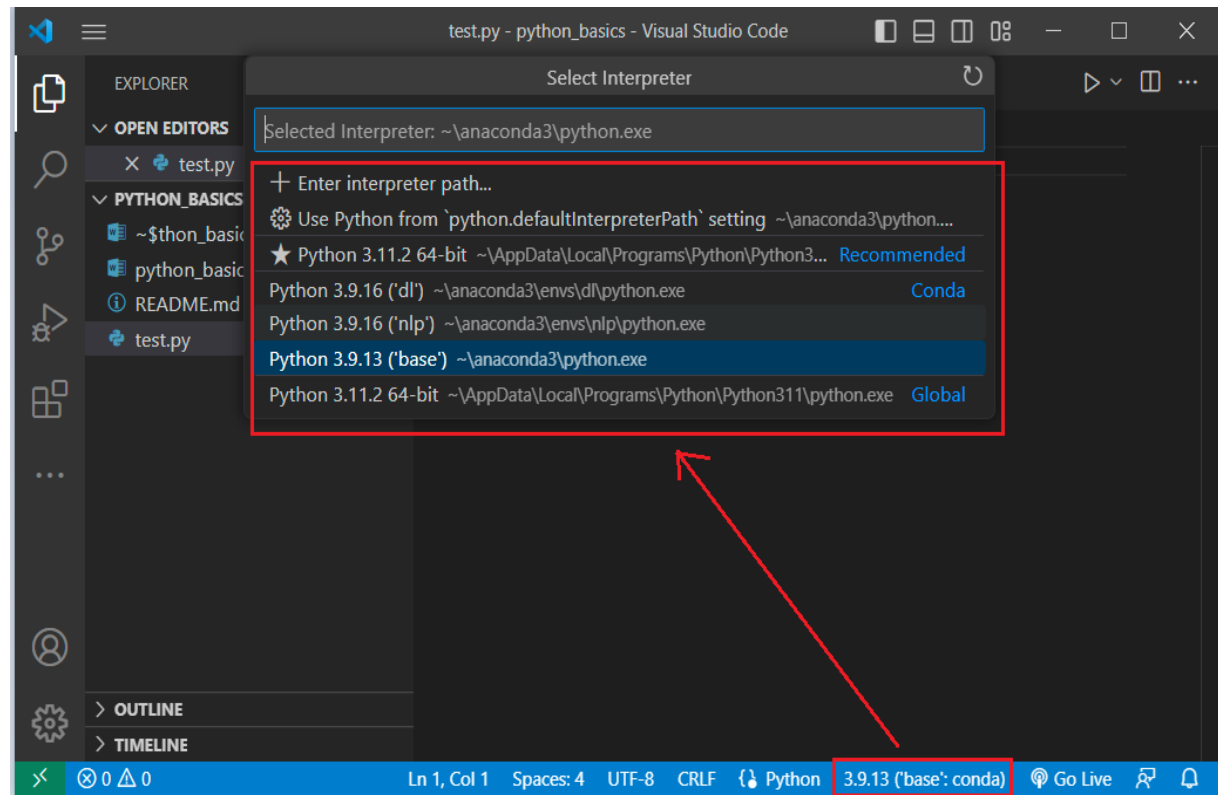
圖：出現 Uninstall 字眼，代表安裝完成，
關閉頁籤，回到專案資料夾



圖：新增檔案，輸入檔案為 test.py，
按下 Enter



圖：開啟 .py 的檔案後，
VS code 自動尋找直譯器



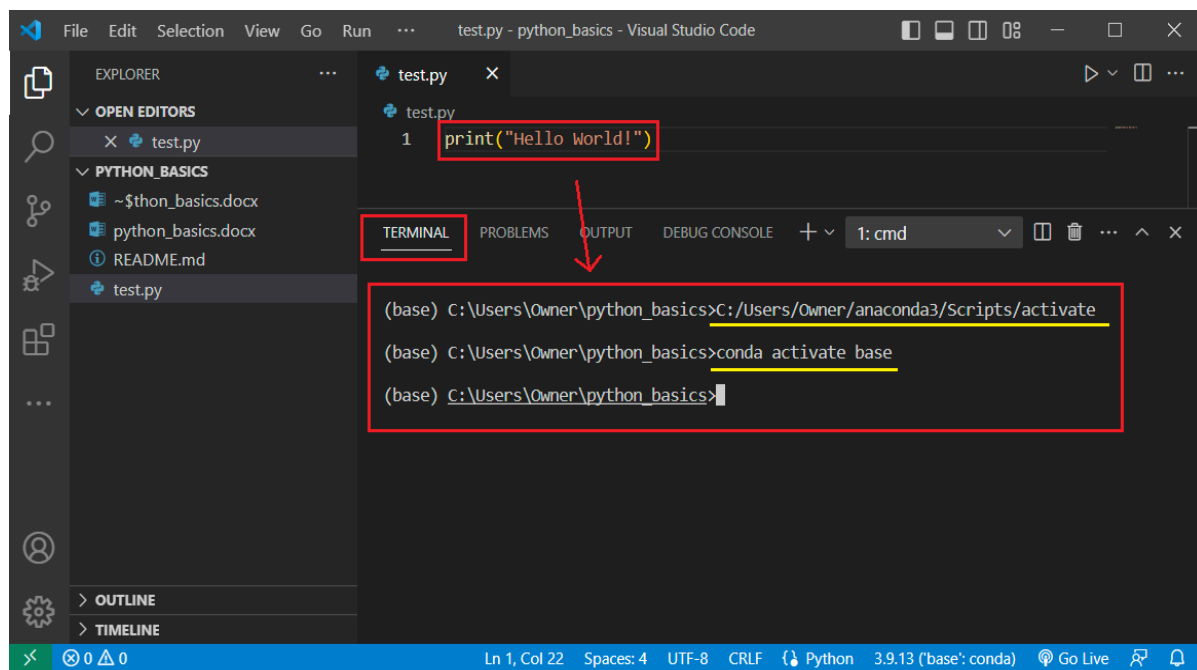
圖：按下右下角的 conda 版本圖示，
可以看到安裝過的環境列表，
可自由切換

建立與執行 .py 檔案

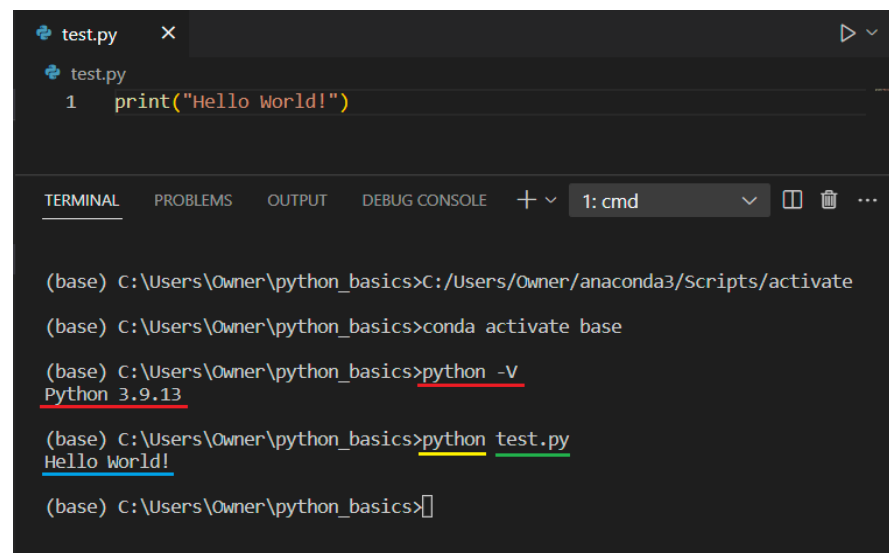
先在剛才的 test.py 檔案中，

輸入「`print( "Hello World!" )`」，

按下快速鍵「`Ctrl + s`」來儲存文字內容 (程式碼)。



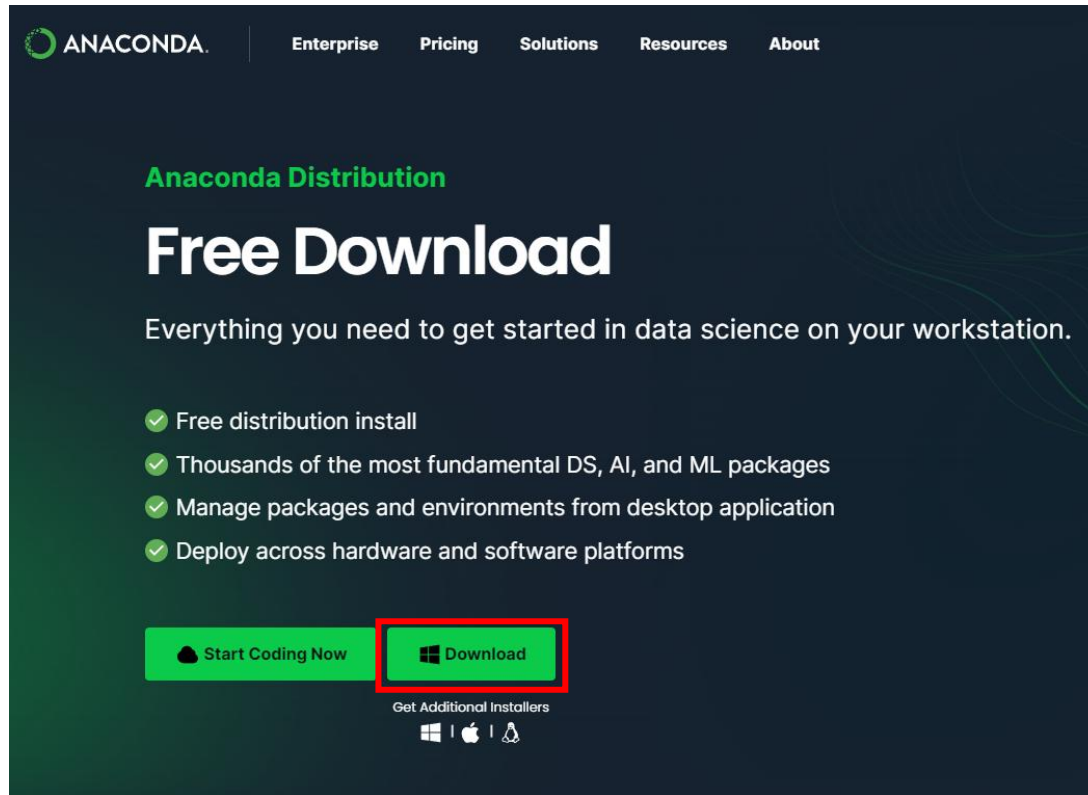
圖：按下快速鍵「`Ctrl + ~`」，顯示終端機 (Terminal)，並啟動 Conda 環境



圖：先確認 base 環境的 python 版本，然後執行 test.py 程式，出現「Hello World!」

安裝Anaconda開發軟體

- Anaconda - Individual Edition
<https://www.anaconda.com/download>

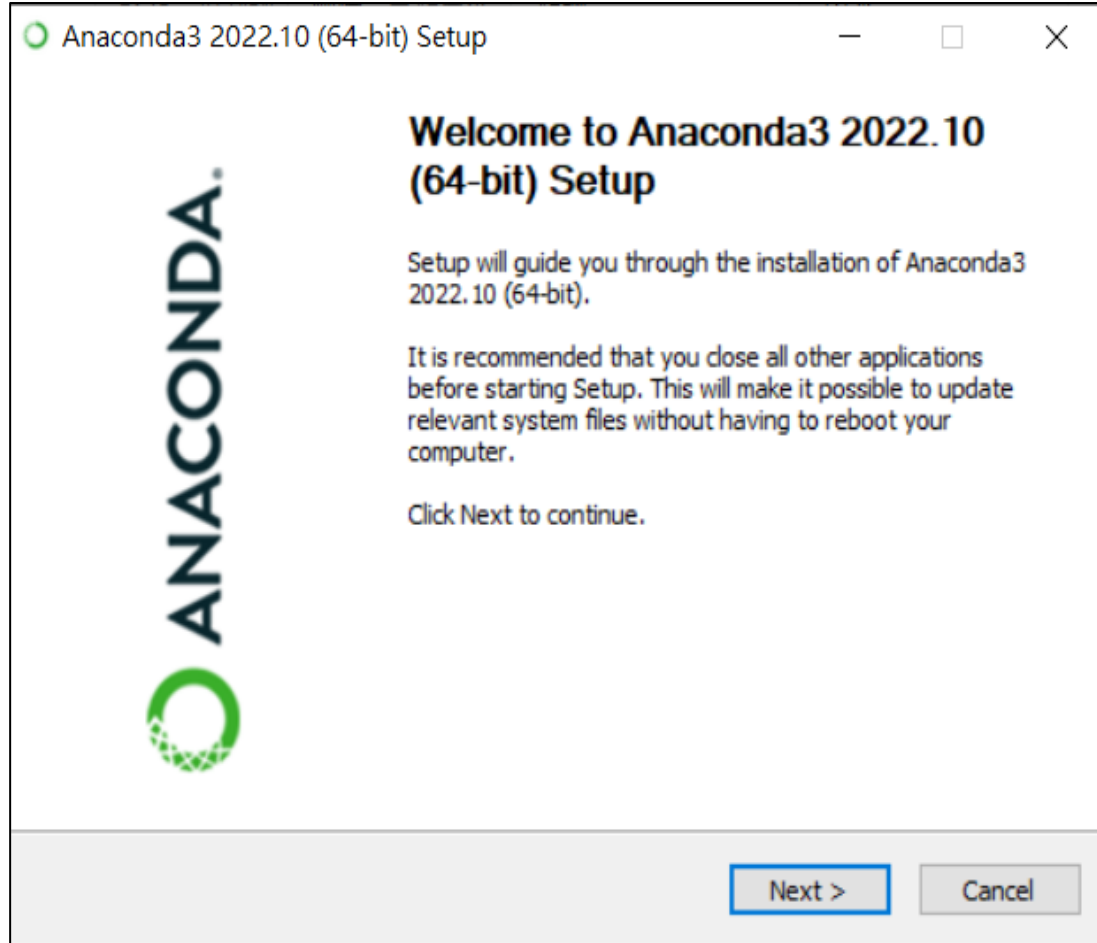


圖：至Anaconda下載頁面，選擇 Download

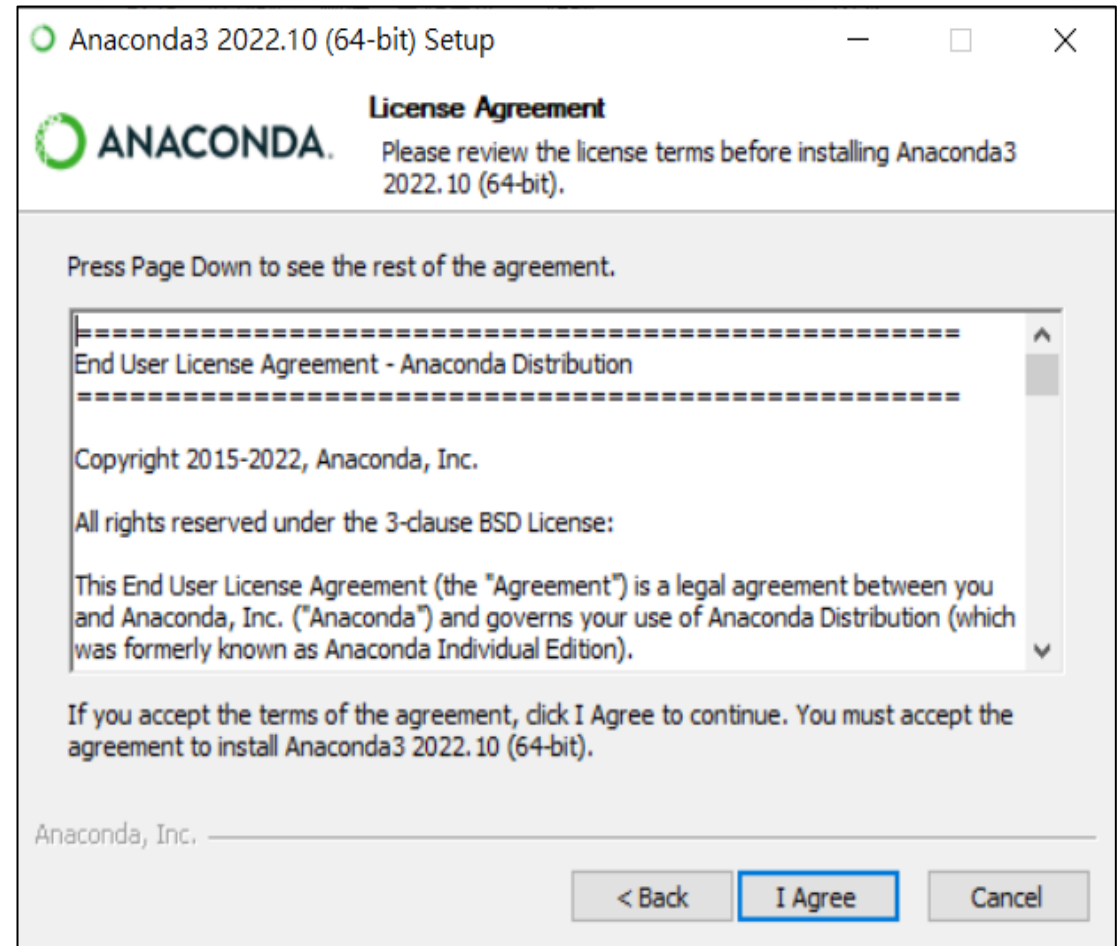


Anaconda3-202
2.10-Windows-x
86_64.exe

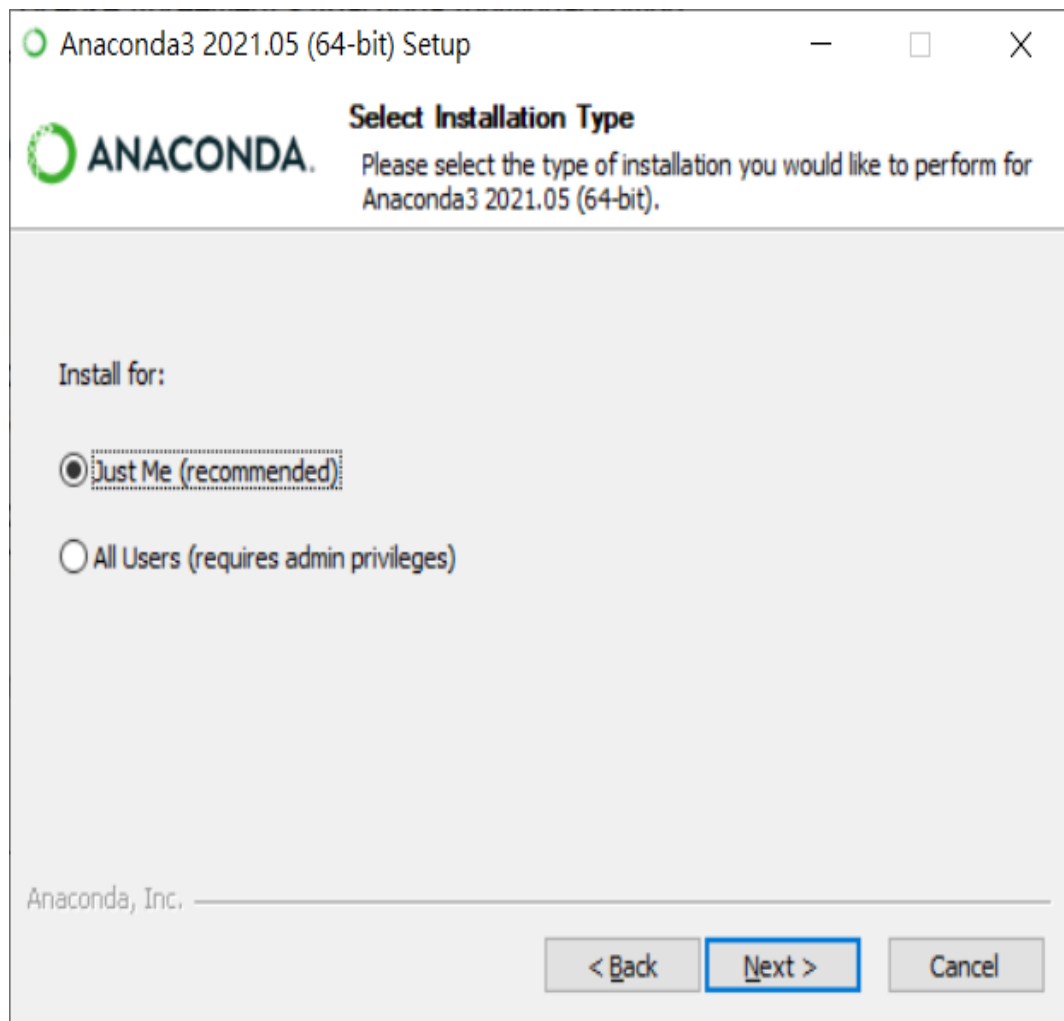
圖：下載後，快速點兩下進行安裝
有安全性警告，可按下執行或同意



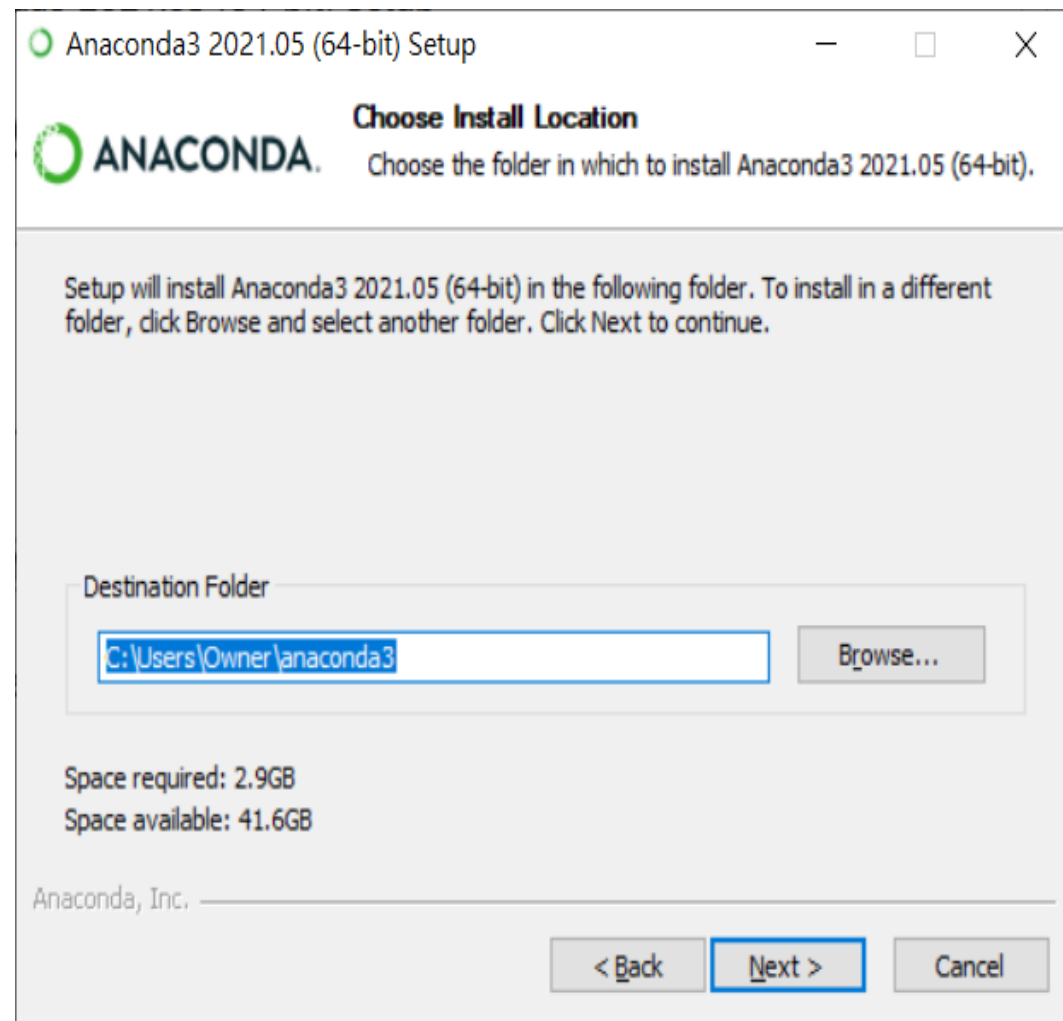
圖：按下「Next」



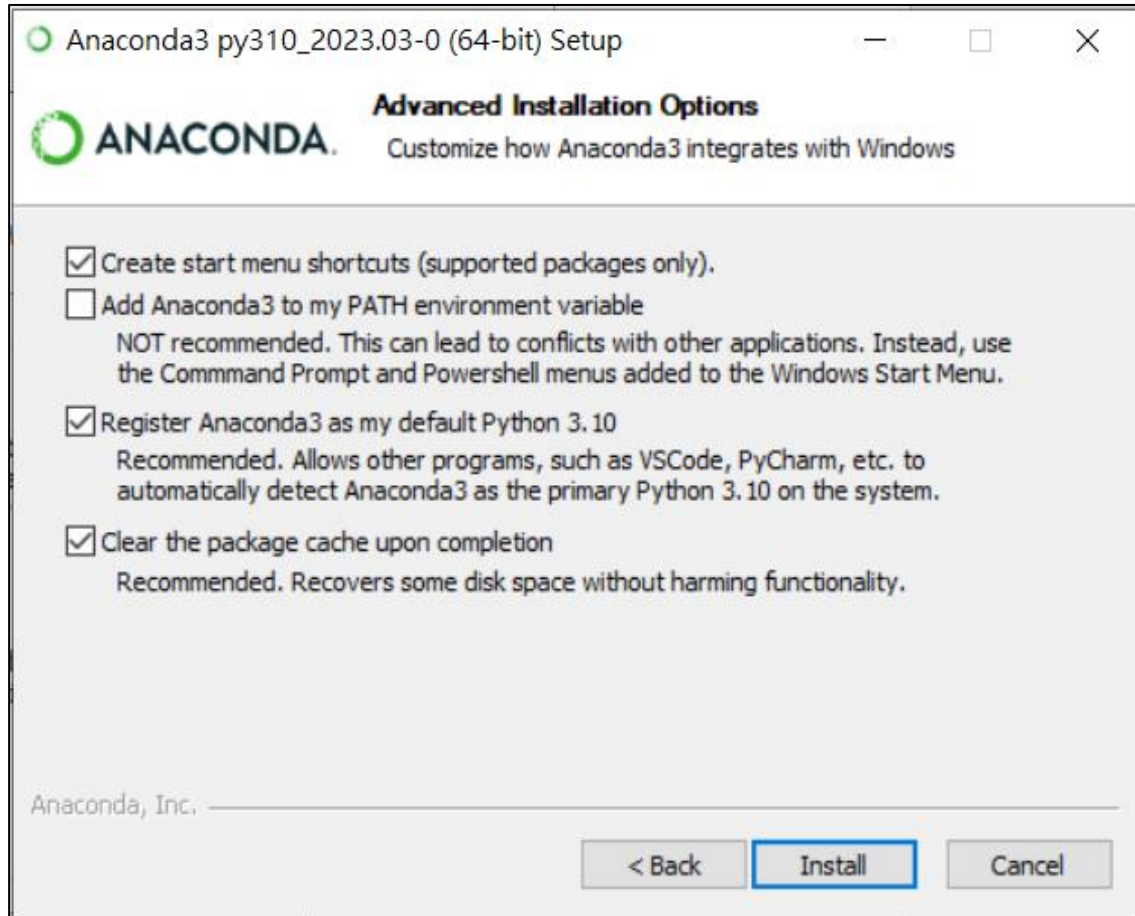
圖：按下「I Agree」



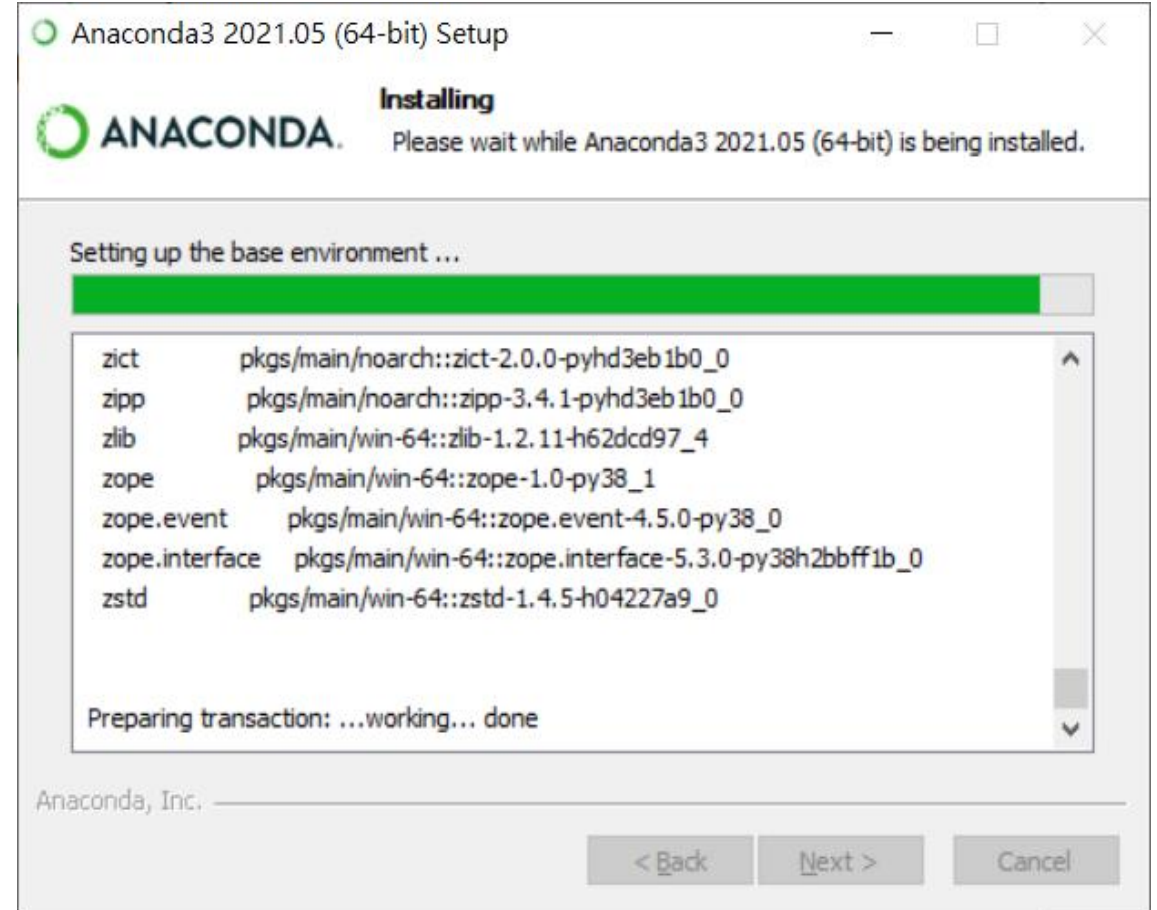
圖：依需求選擇後，按下「Next」



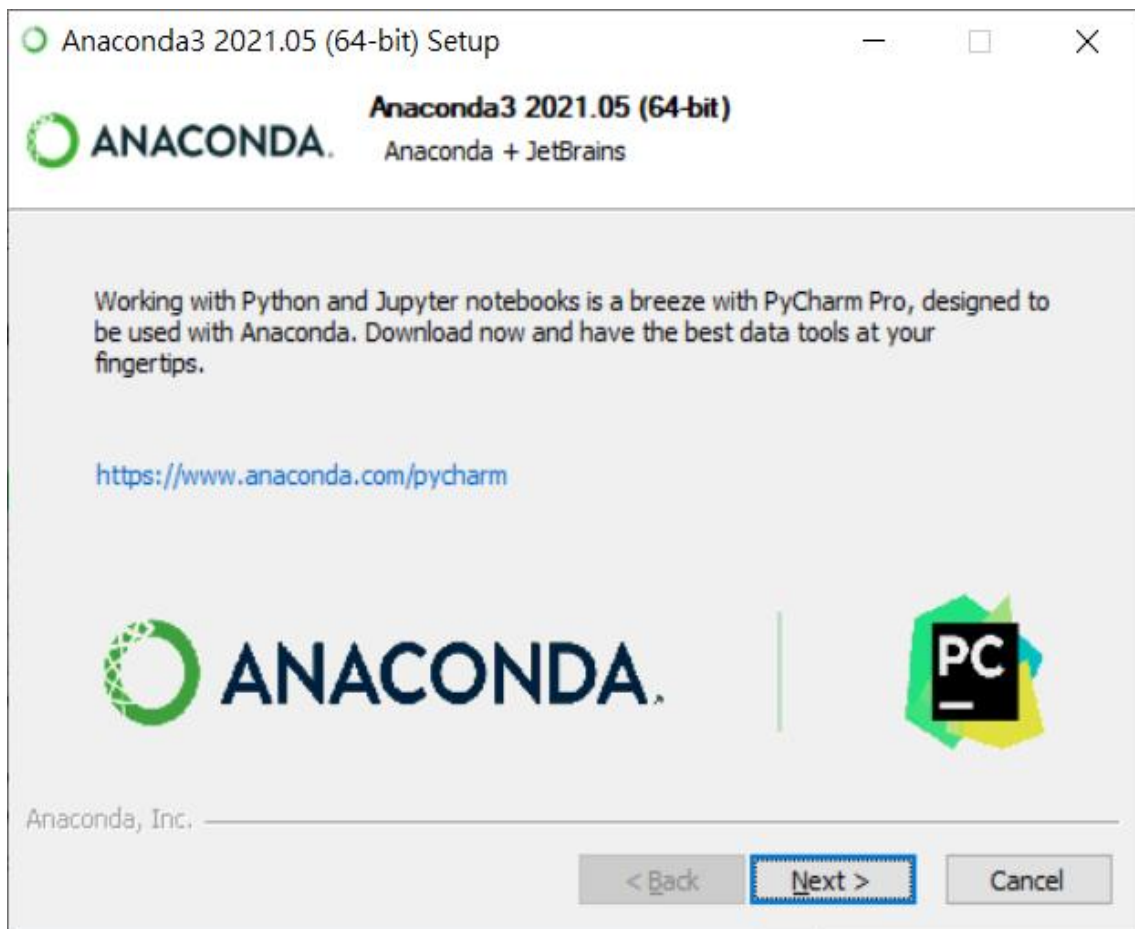
圖：安裝位置會在使用者資料夾中的 anaconda3，依需求設定，按下「Next」



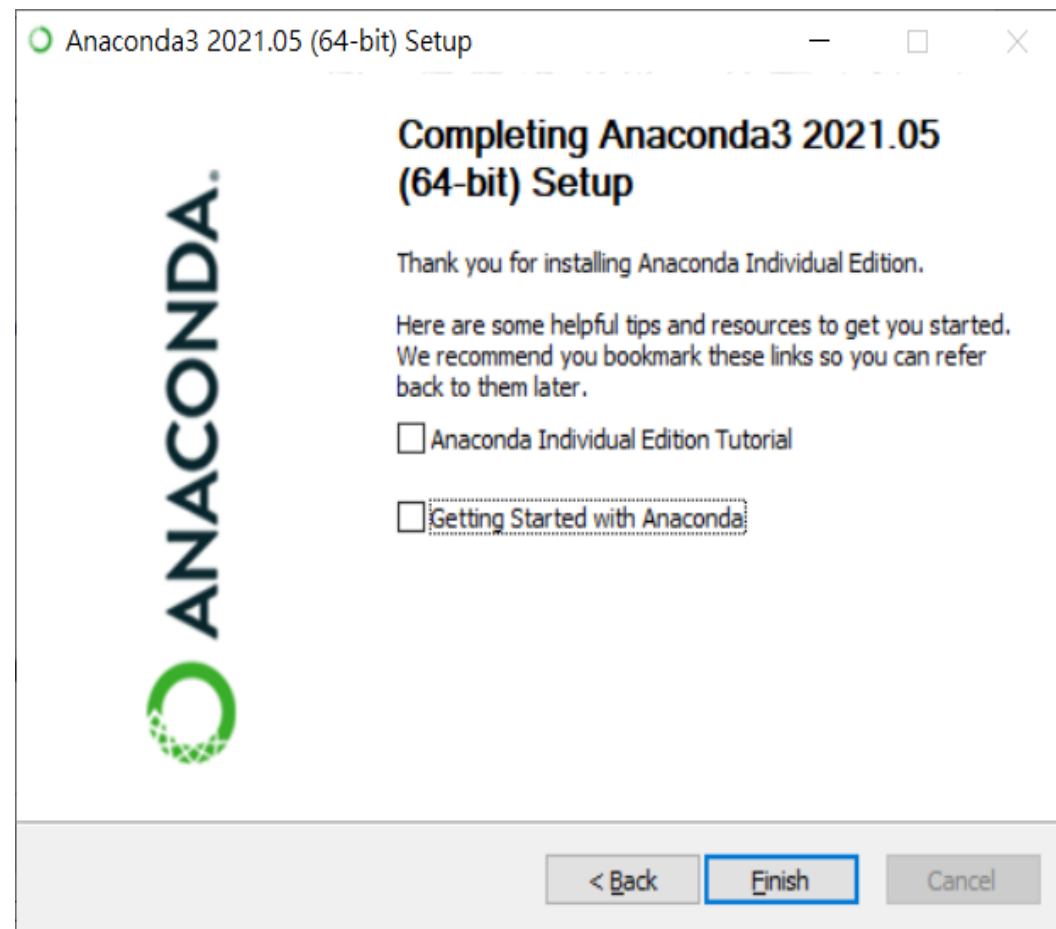
圖：依需求選擇（最後一個建議打勾，省空間），
按下「Install」，進行安裝



圖：按下「Show details」，
會看到安裝過程
完成後按下「Next」



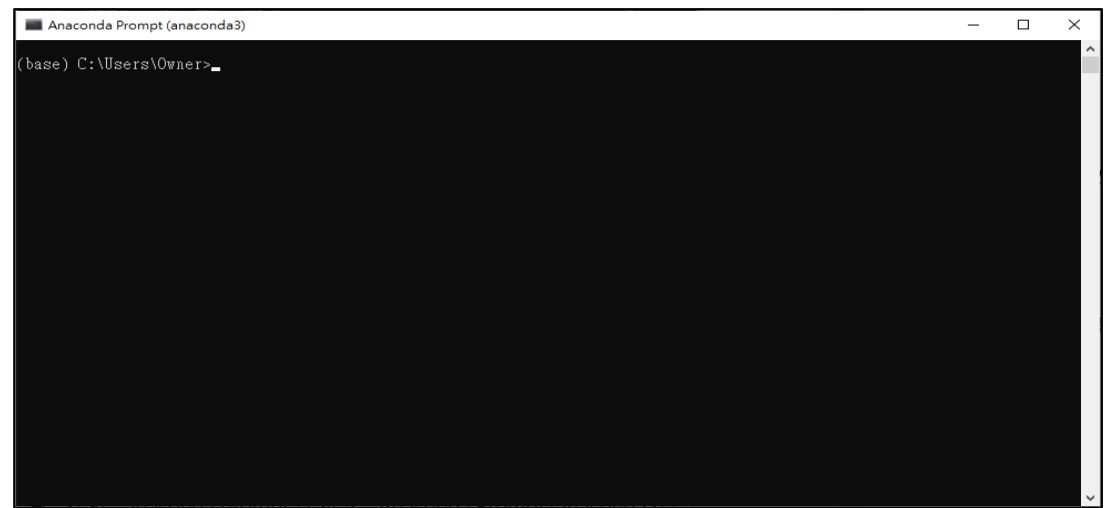
圖：按下「Next」



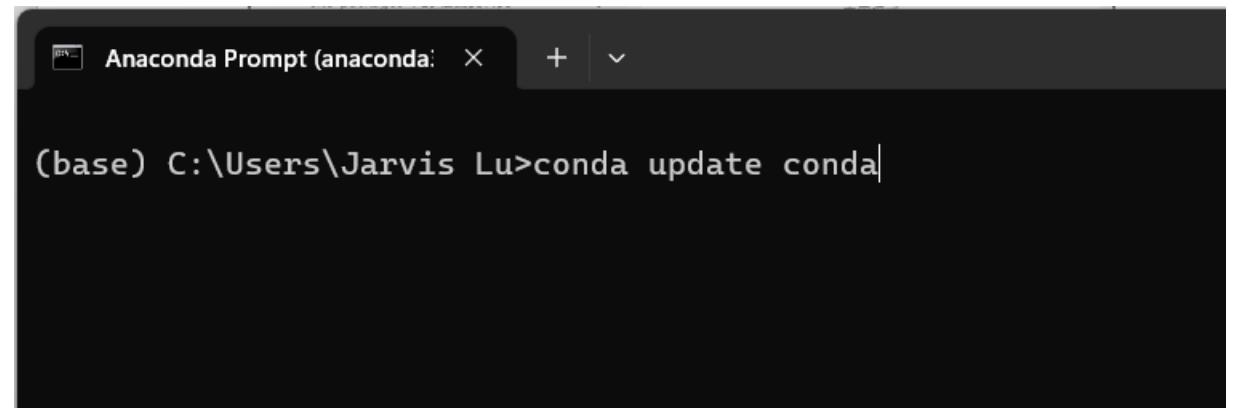
圖：取消勾選圖片中的兩個選項後，
按下「Finish」



圖：搜尋「anaconda prompt」，
按下「Anaconda Prompt (anaconda3)」



圖：出現 Anaconda Prompt，
類似 Windows 的命令提示字元



圖：輸入「conda update conda」，更新 conda 本身

```
Anaconda Prompt (anaconda3)

(base) C:\Users\Owner>conda list
# packages in environment at C:\Users\Owner\anaconda3:
#
# Name                    Version            Build    Channel
_ipyw_jlab_nb_ext_conf    0.1.0              py37_0
alabaster                  0.7.12             py37_0
anaconda                   2020.02            py37_0
anaconda-client            1.7.2              py37_0
anaconda-navigator         1.9.12             py37_0
anaconda-project           0.8.4              py_0
argh                        0.26.2             py37_0
asn1crypto                 1.3.0              py37_0
astroid                    2.3.3              py37_0
astropy                    4.0                py37he774522_0
atomicwrites               1.3.0              py37_1
attrs                      19.3.0             py_0
autopep8                   1.4.4              py_0
babel                      2.8.0              py_0
backcall                   0.1.0              py37_0
backports                  1.0                py_2
backports.functools_lru_cache 1.6.1              py_0
backports.shutil_get_terminal_size 1.0.0            py37_2
backports.tempfile         1.0                py_1
backports.weakref          1.0.post1          py_1
```

圖：輸入「conda list」，
會顯示目前預設安裝的套件
（在虛擬環境base下）

```
Anaconda Prompt (anaconda3)

werkzeug                    1.0.0              py_0
wheel                      0.34.2             py37_0
widgetsnbextension         3.5.1              py37_0
win_inet_pton              1.1.0              py37_0
win_unicode_console        0.5                py37_0
wincertstore               0.2                py37_0
winpty                     0.4.3              4
wrapit                     1.11.2             py37he774522_0
xlrd                        1.2.0              py37_0
xlsxwriter                 1.2.7              py_0
xlwings                    0.17.1             py37_0
xlwt                       1.3.0              py37_0
xmldict                    0.12.0             py_0
xz                          5.2.4              h2fa13f4_4
yaml                       0.1.7              hc54c509_2
yapf                       0.28.0             py_0
zeromq                     4.3.1              h33f27b4_3
zict                       1.0.0              py_0
zipp                       2.2.0              py_0
zlib                       1.2.11             h62dc97_3
zstd                       1.3.7              h508b16e_0

(base) C:\Users\Owner>conda upgrade --all
```

圖：輸入「conda upgrade --all」，
更新當前所有套件

```
Anaconda Prompt (anaconda3) - conda upgrade --all

sphinxcontrib-qth~      1.0.2-py_0 --> 1.0.3-py_0
sphinxcontrib-ser~      1.1.3-py_0 --> 1.1.4-py_0
sphinxcontrib-web~      1.2.0-py_0 --> 1.2.1-py_0
spyder                   4.0.1-py37_0 --> 4.1.3-py37_0
spyder-kernels           1.8.1-py37_0 --> 1.9.1-py37_0
sqlalchemy               1.3.13-py37he774522_0 --> 1.3.16-py37he774522_0
sqlite                   3.31.1-he774522_0 --> 3.31.1-h2a8f88b_1
tornado                  6.0.3-py37he774522_3 --> 6.0.4-py37he774522_1
tqdm                     4.42.1-py_0 --> 4.46.0-py_0
wcwidth                  0.1.8-py_0 --> 0.1.9-py_0
werkzeug                 1.0.0-py_0 --> 1.0.1-py_0
xlswriter                1.2.7-py_0 --> 1.2.8-py_0
xlwings                  0.17.1-py37_0 --> 0.19.0-py37_0
xz                       5.2.4-h2fa13f4_4 --> 5.2.5-h62dcd97_0
zict                     1.0.0-py_0 --> 2.0.0-py_0
zipp                     2.2.0-py_0 --> 3.1.0-py_0
zlib                     1.2.11-h62dcd97_3 --> 1.2.11-h62dcd97_4

The following packages will be DOWNGRADED:

anaconda                 2020.02-py37_0 --> custom-py37_1
lzo                      2.10-h6df0209_2 --> 2.10-he774522_2

Proceed ([y]/n)?
```

圖：按下「y」後，再按鍵盤「Enter」

```
Anaconda Prompt (anaconda3)

- DEBUG menuinst_win32: __init__(199): Menu: name: 'Anaconda${PY_VER} ${PLATFORM}', prefix: 'C:\Users\Owner\anaconda3'
, env_name: 'None', mode: 'user', used_mode: 'user'
DEBUG menuinst_win32:create(323): Shortcut cmd is C:\Users\Owner\anaconda3\pythonw.exe, args are ['C:\Users\Owner\
anaconda3\cwp.py', 'C:\Users\Owner\anaconda3', 'C:\Users\Owner\anaconda3\pythonw.exe', 'C:\Users\Owner\ana
conda3\Scripts\spyder-script.py']
\ DEBUG menuinst_win32:create(323): Shortcut cmd is C:\Users\Owner\anaconda3\python.exe, args are ['C:\Users\Owner\
anaconda3\cwp.py', 'C:\Users\Owner\anaconda3', 'C:\Users\Owner\anaconda3\python.exe', 'C:\Users\Owner\ana
conda3\Scripts\spyder-script.py', '--reset']
done

(base) C:\Users\Owner>
```

圖：套件更新完成

```
Anaconda Prompt (anaconda3) - python
- DEBUG menuinst_win32:_init_(199): Menu: name: 'Anaconda${PY_VER} ${PLATFORM}', prefix: 'C:\Users\Owner\anaconda3'
, env_name: 'None', mode: 'user', used_mode: 'user'
DEBUG menuinst_win32:create(323): Shortcut cmd is C:\Users\Owner\anaconda3\pythonw.exe, args are ['C:\Users\Owner\
anaconda3\cwp.py', 'C:\Users\Owner\anaconda3', 'C:\Users\Owner\anaconda3\pythonw.exe', 'C:\Users\Owner\ana
conda3\Scripts\spyder-script.py']
\ DEBUG menuinst_win32:create(323): Shortcut cmd is C:\Users\Owner\anaconda3\python.exe, args are ['C:\Users\Owner\
anaconda3\cwp.py', 'C:\Users\Owner\anaconda3', 'C:\Users\Owner\anaconda3\python.exe', 'C:\Users\Owner\ana
conda3\Scripts\spyder-script.py', '--reset']
done

(base) C:\Users\Owner>python
Python 3.7.7 (default, May 6 2020, 11:45:54) [MSC v.1916 64 bit (AMD64)] :: Anaconda, Inc. on win32
Type "help", "copyright", "credits" or "license" for more information.
>>>
```

圖：輸入「python」，進入 python 執行環境

```
Anaconda Prompt (anaconda3) - python
- DEBUG menuinst_win32:_init_(199): Menu: name: 'Anaconda${PY_VER} ${PLATFORM}', prefix: 'C:\Users\Owner\anaconda3'
, env_name: 'None', mode: 'user', used_mode: 'user'
DEBUG menuinst_win32:create(323): Shortcut cmd is C:\Users\Owner\anaconda3\pythonw.exe, args are ['C:\Users\Owner\
anaconda3\cwp.py', 'C:\Users\Owner\anaconda3', 'C:\Users\Owner\anaconda3\pythonw.exe', 'C:\Users\Owner\ana
conda3\Scripts\spyder-script.py']
\ DEBUG menuinst_win32:create(323): Shortcut cmd is C:\Users\Owner\anaconda3\python.exe, args are ['C:\Users\Owner\
anaconda3\cwp.py', 'C:\Users\Owner\anaconda3', 'C:\Users\Owner\anaconda3\python.exe', 'C:\Users\Owner\ana
conda3\Scripts\spyder-script.py', '--reset']
done

(base) C:\Users\Owner>python
Python 3.7.7 (default, May 6 2020, 11:45:54) [MSC v.1916 64 bit (AMD64)] :: Anaconda, Inc. on win32
Type "help", "copyright", "credits" or "license" for more information.
>>> print("Hello, world")
Hello, world
>>>
```

圖：輸入「print("Hello, world")」，
輸出「Hello, world」

安裝、切換與刪除 Conda 環境 (Env)

- 預設是 (base)，如果有切換環境的需求，例如手上處理著不同 Python 版本和其它相關套件的專案，需要不時切換版本來開發，此時可以建立一到多個 Conda 環境，需要的時候可以切換，不需要的時候可以刪除。
- 在執行以下的指令前，需要確認目前是否在 Anaconda Prompt 當中，或是支援 Conda 的 Terminal 環境。終端機顯示預設路徑時，最前面會有 (base)，代表目前正在預設的 Conda 環境當中。

Conda 指令

- 安裝 Conda 環境
 - **conda create --name <自訂環境名稱> [python=版本號]**
 - conda create --name python_basic python=3.10
- 列出所有 Conda 環境
 - conda env list
- 離開當前的 Conda 環境
 - conda deactivate
- 切換 / 啟動 Conda 環境
 - conda activate python_basic
- 刪除 Conda 環境
 - conda env remove -n python_basic

Module 2.

資料型態與變數

變數

- 變數 (variable) 就是一個**暫存資料 (值)** 的地方，會將資料儲存在電腦記憶體當中，原則上什麼值都給可以賦予、分配 (assign) 給它。使用明確、描述性的變數名可以使你的程式更易讀、更易理解。

2-1.py

```
x = 1
name = "Alex"

print(x)
print(name)
```

變數命名規則、關鍵字

- 變數命名規則

- 由字母 (A-z)、數字 (0-9) 符號和底線 () 組成。
- 必須以字母 (A-z) 或底線 () 開頭，**不能以數字開頭**。
- 區分大小寫，name、Name、NAME 是不同的變數。

```
myvar = "John"  
my_var = "John"  
_my_var = "John"  
myVar = "John"  
MYVAR = "John"  
myvar2 = "John"
```

圖：正確的變數命名方式

```
2myvar = "John"  
my-var = "John"  
my var = "John"
```

圖：錯誤的變數命名方式

• 關鍵字

- 所謂關鍵字 (keywords)，是用來保留給程式語言使用的內建語法或是變數名稱，不適合作為一般變數來使用，以下列出常見的 Python 關鍵字：

False	await	else	import	pass
None	break	except	in	raise
True	class	finally	is	return
and	continue	for	lambda	try
as	def	from	nonlocal	while
assert	del	global	not	with
async	elif	if	or	yield

圖：參考官方文件 https://docs.python.org/3/reference/lexical_analysis.html#keywords

• 註解

- 註解 (comments) 在程式碼當中，通常是用於對撰寫的程式碼進行文字說明，或是希望其它行的程式碼暫時不要執行，先用單行註解或多行註解先將其關閉，待需要的時候才打開。

單行註解	多行註解	多行註解
# 單行註解	''' 多行註解 多行註解 多行註解 '''	'''' 這也是多行註解 這也是多行註解 這也是多行註解 ''''

註：

在 VS code 當中，可以反白幾行程式碼，之後再對它們按下 Ctrl + /，可以作到多個單行註解；如果要解除註解，對原先註解的程式碼進行反白，再按下 Ctrl + /。

資料型態

資料型態	說明	範例
整數 (Integer)	基本的數據類型，代表整數	<code>x = 10</code>
浮點數 (Float)	代表具有小數點的數字	<code>y = 2.5</code>
布林值 (Boolean)	用來表示真(True)或假(False) 在進行條件判斷或者邏輯運算時使用	<code>parse_ok = True</code>
字串 (String)	代表文字數據。你可以用單引號或雙引號來定義一個字串	<code>message = 'Hello'</code> <code>name = "John"</code>
列表 (List)	容納多個值，這些值可以是任意數據類型，並且可修改。 使用中括號「 <code>[]</code> 」將資料整理在一起。	<code>my_list = [1, 2, 3, 4]</code>
元組 (Tuple)	元組與列表類似，但是一旦創建就不能修改。 使用小括號「 <code>()</code> 」將資料整理在一起。	<code>my_tuple = (1, 2, 3)</code>
集合 (Set)	類似 list 和 tuple，但它儲存的資料「不能重複」。 使用大括號「 <code>{}</code> 」將資料整理在一起。	<code>my_set = {1, 2, 3}</code>
字典 (Dictionary)	是一種鍵值對 {key:value} 的數據結構。每個元素都有一個鍵和一個值，可以透過鍵(key)來查找對應的值(value)。	<code>data = {'name': 'John', 'age': 5}</code>

運算子

- 算術運算子 (Arithmetic operators)
- 賦值運算子 (Assignment operators)
- 比較運算子 (Comparison operators)
- 邏輯運算子 (Logical operators)
- 成員運算子 (Membership operators)
- 身分運算子 (Identity operators)
- 位元運算子 (Bitwise Operators)

• 算術運算子

運算符號	說明	範例
+	加	9 + 4 回傳 13
-	減	9 - 4 回傳 5
*	乘	9 * 4 回傳 36
/	除	9 / 4 回傳 2.25
%	相除取得餘數	9 % 4 回傳 1
**	多少的幾次方	9 ** 4 回傳 6561
//	相除取得整數	9 // 4 回傳 2

- 賦值運算子 $a = 3, b = 4$

運算符號	說明	範例
=	單純賦值 (值由右往左給)	$c = a + b$ c的值為 7
+=	計算加法後賦值	$b += a$, 等同於 $b = b + a$ b的值為 7
-=	計算減法後賦值	$b -= a$, 等同於 $b = b - a$ b的值為 1
*=	計算乘法後賦值	$b *= a$, 等同於 $b = b * a$ b的值為 12
/=	計算除法後賦值	$b /= a$, 等同於 $b = b / a$ b的值為 1.3333333333333333
%=	計算取餘數後賦值	$b \% = a$, 等同於 $b = b \% a$ b的值為 1
**=	計算幾次方後賦值	$b ** = a$, 等同於 $b = b ** a$ b的值為 64
//=	計算取得整數後賦值	$b //= a$, 等同於 $b = b // a$ b的值為 1

• 比較運算子

運算符號	說明	範例
==	等於	5 == 5 回傳True
!=	不等於	5 != 6 回傳True
>	大於	5 > 6 回傳 False
<	小於	5 < 6 回傳 True
>=	大於等於	5 >= 6 回傳 False
<=	小於等於	5 <= 6 回傳 True

• 縮排

- Python 程式語言中，縮排不僅是為了使程式碼看起來更整齊、更易讀，更重要的是，它在 Python 中被用來表示程式結構
- 某一行以關鍵字 (`if`、`for`、`try`) 開始，以冒號「:」結尾，下一行就要縮排，來代表程式區塊的開始
- Python 建議使用 **4 個空格**來表達下一個區塊

<code>score = 50</code> <code>passing_score = 60</code>	宣告 <code>score</code> 變數的值為 50 宣告 <code>passing_score</code> 變數的值為 60
<code>if score < passing_score:</code>	條件語句的開始，用「:」結尾
<code> score = score + 10</code> <code> print('加分')</code>	程式碼前放了 4 個空格 () 縮排，代表 在同一個程式區塊內。
(下一行程式) (下一行程式)	如果下一行程式前面還有縮排， 代表都在同一個程式區塊範圍內。
<code>print(score)</code>	這是下一行程式，不受上方程式區塊的範圍影響

• 邏輯運算子

運算符號	說明	範例
and	若是兩邊都是 True，則回傳 True	<pre>name = 'Alex' age = 18 if name == 'Alex' and age == 18: print('正確')</pre>
or	若是兩邊其中一個是 True，則回傳 True	<pre>name = 'Alex' age = 18 if name == 'Alex' or age == 16: print('正確')</pre>
not	若變數的值是 True，則回傳 False；若變數的值是 False，則回傳 True	<pre>flag = False if not(flag): print('從 False 變成 True')</pre>

• 成員運算子

運算符號	說明	範例
in	檢查某個元素是否存在於某個容器中，在則回傳 True，反之則回傳 False。	<pre>myList = [9, 5, 2, 7] if 2 in myList: print('2 在 list 當中')</pre>
not in	檢查某個元素是否不存在於某個容器中，不在則回傳 True，在則回傳 False。	<pre>myList = [9, 5, 2, 7] if 3 not in myList: print('3 不在 list 當中')</pre>

• 身分運算子

運算符號	說明	範例
is	當兩個變數 <u>是</u> 參照同一個物件(同一個記憶體位置)，回傳 True。	<pre>x = 123 y = x # y 參照 x 的記憶體位置 if x is y: print('x 和 y 是同一物件')</pre>
is not	當兩個變數 <u>不是</u> 參照同一個物件(同一個記憶體位置)，回傳 True。	<pre>x = 123 y = 456 # y 隨機佔用記憶體位置 if x is not y: print('x 和 y 不是同一物件')</pre>

• 位元運算子

提示：位元運算會將運算元轉成二進制，再進行運算元之間的布林運算

運算符號	說明	範例
&	AND 運算 (相同為1，就是1)	<pre>x = 1 y = 3 print(x & y) # 輸出 1</pre>
	OR運算 (其中為1，就是1)	<pre>x = 1 y = 3 print(x y) # 輸出 3</pre>
^	XOR運算 (相同為0，相異為1)	<pre>x = 1 y = 3 print(x ^ y) # 輸出 2</pre>
~	補數運算 (有正負號，運算才有義意)	<pre>x = +3 print(~x)# 輸出 -4</pre>
<<	位元左移	<pre>x = 4 print(x << 1) # 輸出 8</pre>

• 運算子優先順序

運算子	說明
()	括號
**	冪、指數
+X -X ~X	一元運算子加號、一元運算子減號、位元運算的 not
* / // %	乘、除、除(只取商數)、相除取餘數
+ -	加、減
<< >>	位元左移、位元右移
&	位元運算 AND
^	位元運算 XOR
	位元運算 OR
== != > >= < <= is is not in not in	比較運算子 身分運算子 成員運算子
not	(邏輯運算)True變成False，或是 False 變成 True
and	(邏輯運算)且
or	(邏輯運算)或

2-3.py

```
# 算術運算子
print(9 + 4)
print(3 / 2)
print(2 ** 3)

# 賦值運算子
a, b = 9, 4
c, d = 4, 2
a += b # 等於 a = a + b
print(a)
d **= c # 等於 d = d ** c
print(d)

# 比較運算子 (用 if 判斷)
e = 4
if e == 4:
    print('e 等於 4')
if e > 2:
    print('e 大於 2')
if e != 5:
    print('e 不等於 5')
```

```
# 邏輯運算子
f = 8
name = 'Bill'
if f == 8 and name == 'Bill':
    print('ok')
if f == 6 or name == 'Bill':
    print('name 等於 Bill')

# 成員運算子
myList = [5, 5, 6, 6, 3, 3, 1, 2]
if 6 in myList:
    print('6 在 list 裡')
if 4 not in myList:
    print('4 不在 list 裡')
```

Module 3.

控制結構

if...else 、 for 、 while

控制結構

Python 的**控制結構**是用來控制程式「執行流程」的工具，它決定程式要：

- 依條件選擇要不要執行 → 條件語句 (if-elif-else)
- 重複執行某段程式 → 迴圈 (for 、 while)
- 提早中斷或跳過某次執行 → 迴圈控制語句 (break 、 continue)

控制結構

1. 條件語句 (**if-elif-else**) : 條件語句用來進行條件判斷，只有滿足某個條件，相對應的程式碼區塊才會被執行。

- **if**

在右方敘述包含一個結果為 Boolean 的運算，下方伴隨一個程式碼區塊

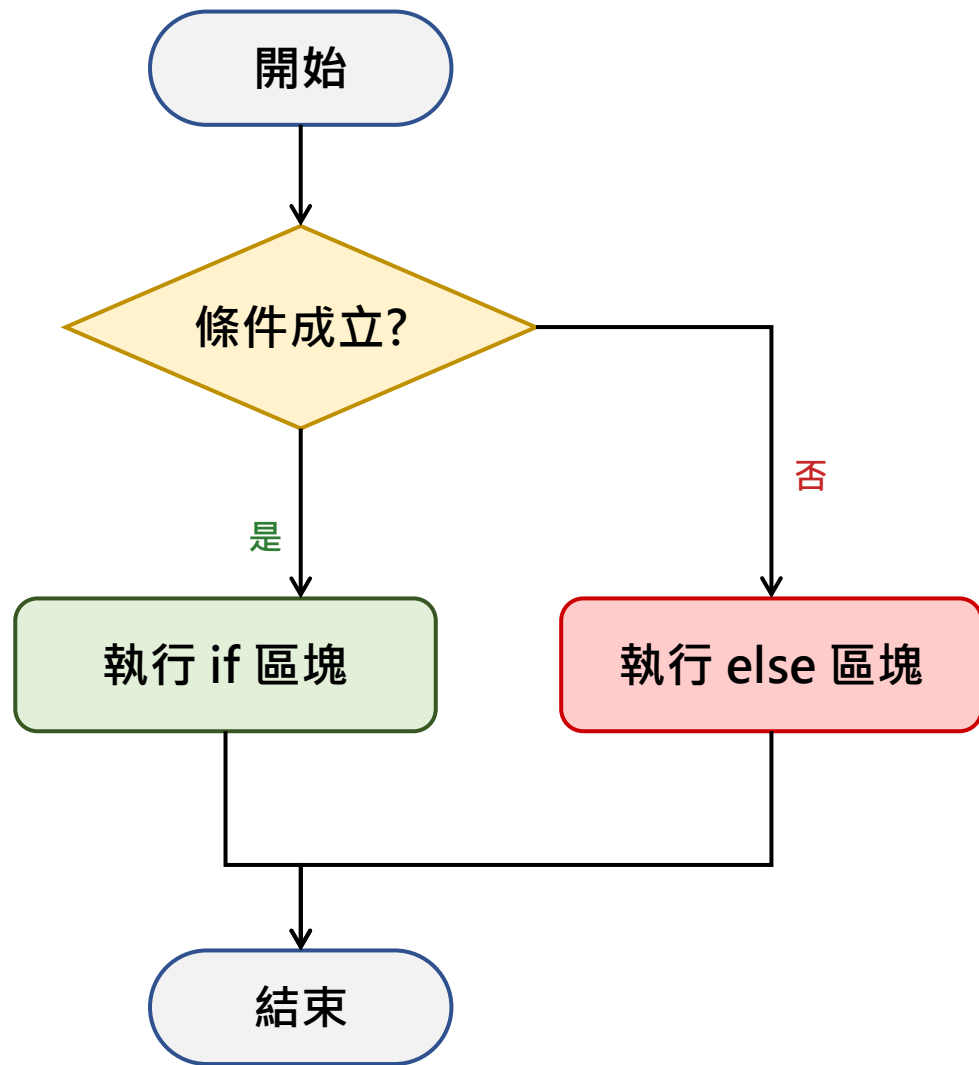
- **if...else**

如果 if 敘述的判斷式為真，則執行 if 後面的敘述，否則執行 else 後面的敘述

- **if...elif...else**

如果 if 敘述為 False，就照順序判斷所有 elif 敘述，都為 False 才執行 else

if...else 執行流程



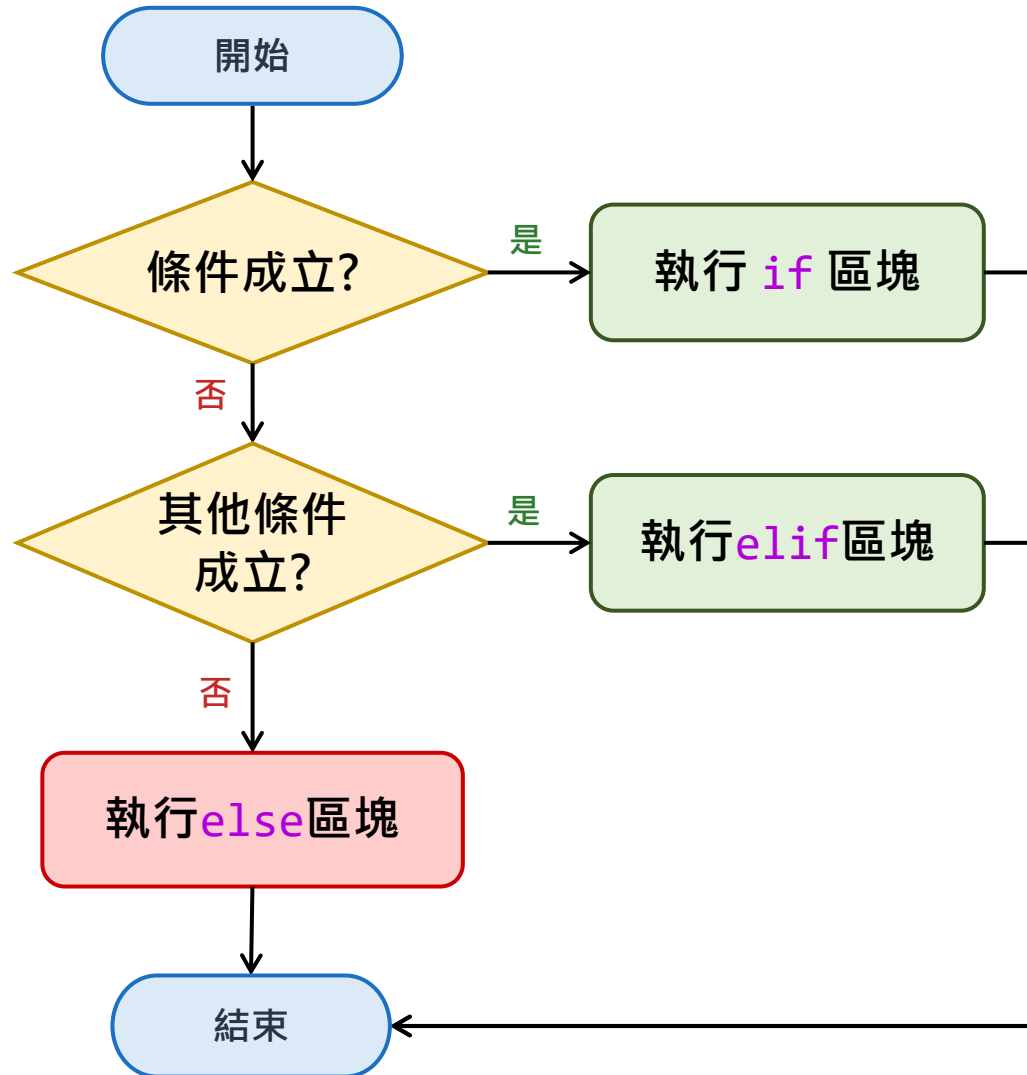
程式碼範例

```
score = 75
if score >= 60:
    print("及格!")
else:
    print("不及格!")
```

執行結果：

及格！

if...elif...else 執行流程



程式碼範例

```
score = 95
if score == 100:
    print("滿分!")
elif score >= 90:
    print("優秀!")
elif score >= 60:
    print("及格!")
else:
    print("不及格!")
```

執行結果：

優秀！



只要有其中一個符合，
後面的就不判斷
在使用 if...elif... 時，
需要特別留意順序!!

3-1.py

```
# if
num = 10

if num > 5:
    print("num 大於 5")

# if else
name = 'apple'
if name == 'apple':
    print('name is apple')
else:
    print("name is not apple")

# if elif (else)
name = 'Jarvis'
if name == "alex":
    print("name: alex")
elif name == "bill":
    print("name: bill")
```

2. 迴圈 (for 、 while) : 迴圈用來進行重複的操作。

- **for** 迴圈

通常用來遍歷一個序列 (如列表 List 或 字串 String)

- **while** 迴圈

在滿足某個條件的情況下不斷循環直到該條件變為 **False** 。

3. 迴圈控制語句 (break 、 continue) : 用來改變迴圈的執行流程。

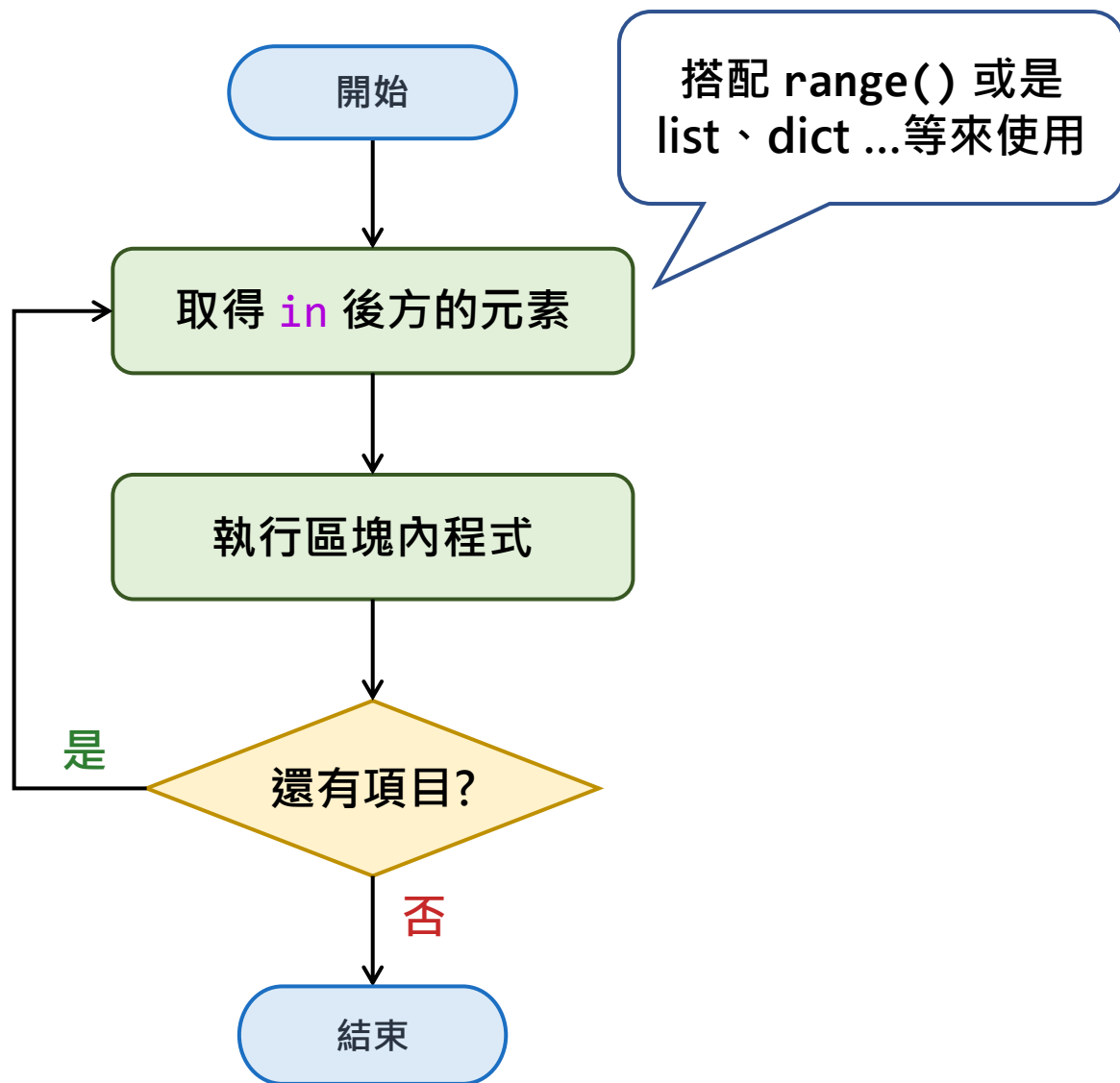
- **break**

立即結束當前迴圈(不論剩幾次)

- **continue**

跳過當前迴圈的剩餘部分(continue 以下的程式碼)，直接進入下一輪迴圈。

for 迴圈 執行流程



程式碼範例

```
for i in range(3):  
    print(i)
```

執行流程：

迴圈第 1 次執行 → 輸出 0
迴圈第 2 次執行 → 輸出 1
迴圈第 3 次執行 → 輸出 2
(沒有更多項目 → 結束)

執行結果：

0
1
2

range() - 產生一串整數 搭配 for 使用

基本語法

`range(stop)`

`range(start, stop)`

`range(start, stop, step)`



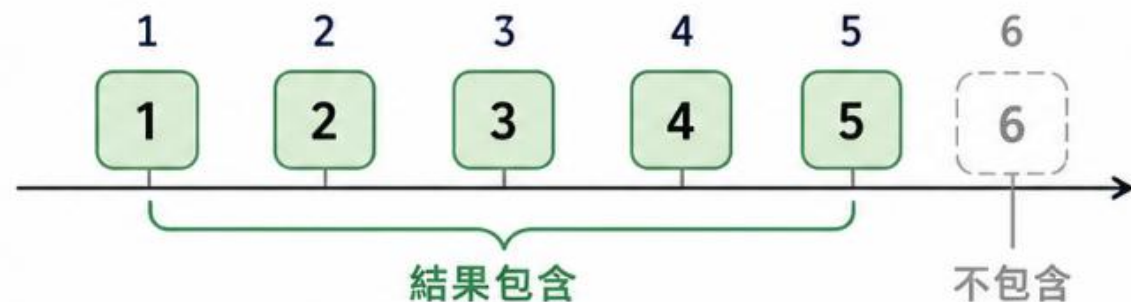
重點：**stop 不包含** 在結果裡！

用途

- ✓ 重複執行固定次數
- ✓ 依序建立數字(由0起算)
- ✓ 搭配 `for` 迴圈走訪指定數字範圍

視覺化說明

以 `range(1, 6)` 為例 → 產生 1~5 (不包含 6)



程式碼範例

```
for i in range(1, 6):  
    print(i, end=" ", " ")
```

執行結果：

1, 2, 3, 4, 5

- 使用 **for** 迴圈遍歷可迭代的(iterable)資料結構(Ex: Str)

程式碼範例

```
for char in 'contents':  
    print(char, end=", ")  
# 輸出 c, o, n, t, e, n, t, s
```

1. 迴圈第一次執行，將 **c** 賦值給 **char** 這個變數

2. 迴圈第二次執行，將 **o** 賦值給 **char** 這個變數

3. 迴圈第三次執行，將 **n** 賦值給 **char** 這個變數

⋮

一直執行到 **s** 結束



for char in 'contents':



for char in 'contents':



for char in 'contents':

- 使用 **for** 迴圈遍歷可迭代的(iterable)資料結構(Ex: List)

程式碼範例

```
for num in [1, 2, 3, 4, 5]:  
    print(num, end=", ")  
# 輸出 1, 2, 3, 4, 5,
```

1. 迴圈第一次執行，將 1 賦值給 num 這個變數

2. 迴圈第二次執行，將 2 賦值給 num 這個變數

3. 迴圈第三次執行，將 3 賦值給 num 這個變數

⋮

一直執行到 5 結束



for num in [1, 2, 3, 4, 5]:

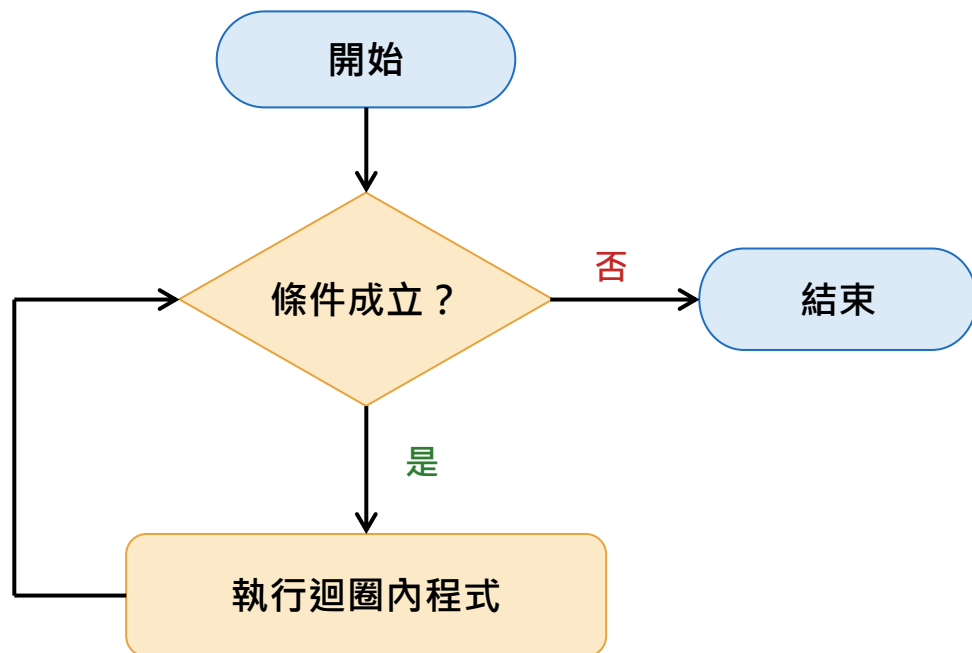


for num in [1, 2, 3, 4, 5]:



for num in [1, 2, 3, 4, 5]:

while 迴圈 執行流程



程式碼範例

```
i = 1
while i <= 3:
    print(i, end=" ", " ")
    i += 1
```

執行流程：

i = 1 → 輸出 1
i = 2 → 輸出 2
i = 3 → 輸出 3
i = 4 → 條件不成立 → 結束

執行結果：

1, 2, 3

3-2.py

for 迴圈

'''

用法

range(n, m-1)

說明

會走訪 n 到 m-1 的數字

'''

```
for i in range(5, 8):  
    print(i, end = ",")
```

while 迴圈

count = 1

```
while count <= 5:  
    print(count, end="")  
    count = count + 1 # 或是寫成 count += 1
```

3-3.py

break

'''

當偵測到字母 t 時，就會強制結束迴圈

'''

```
for char in 'content':  
    if char == 't':  
        break  
    print(char, end="")
```

continue

'''

當偵測到字母 t 時，
會跳過本次迴圈剩下的程式碼 print(char)，
但不會結束迴圈，仍然會進入下一圈繼續執行

'''

```
for char in 'content':  
    if char == 't':  
        continue  
    print(char, end="")
```

補充：input()

- 若是希望能在程式執行期間，讓使用者可以自行輸入資料 (類似提示對話視窗 prompt)，之後整合到程式碼之後輸出結果，可以使用 input() 函式。

3-4.py

```
# 取得使用者輸入的資料
data = input('輸入文字後，按下 Enter: ')
print(data)

# 將使用者輸入強制轉型成 int
num = int(input('請輸入數字: '))
print(type(num))
print(num)
```


Module 4.

字串、列表、元組、 字典、集合

字串 str

- 字串需要用「兩個單引號」或「兩個雙引號」包起來，
例如 `'星期天'` 或 `"星期天"`。
- 字串是不可變的，無法更改已經創建字串中的單個字元。
- 有索引 (index)，可通過索引來取得相對應的字元
- 這個存取的代號 0、1、2...等等，稱之為「索引」(index)。

索引值：	0	1	2	3	4	5	6	7	8	9	10	11
	'H'	'e'	'l'	'l'	'o'	','	'W'	'o'	'r'	'l'	'd'	'l'
索引值：	-12	-11	-10	-9	-8	-7	-6	-5	-4	-3	-2	-1

字串 str 介紹

什麼是 str ?

- str 是一種「字串」資料型態
- 用單引號 ' ' 或雙引號 " " 包住文字
- 字串是不可變的 (immutable)

字串由字元組成

h	e	l	l	o
0	1	2	3	4

基本語法

```
name = "Python"  
msg = '學習很有趣!'
```

- ✓ 用單引號 ' ' 或雙引號 " "
- ✓ 結果都是字串 (str)
- ✓ 字串不可變 (immutable)

常見操作

	讀取：	name[0]	→	P
	切片：	name[1:4]	→	yth
	長度：	len(name)	→	6
	串接：	"Py" + "thon"	→	Python
	重複：	"ha" * 3	→	hahaha

程式碼範例

```
text = "Python"  
print(text[0])  
print(text[1:4])  
print(len(text))
```

執行結果

```
P  
yth  
6
```



重點提醒

- ✓ str 可用索引取得字元
- ✓ 支援切片與許多字串方法
- ✓ 內容不可直接修改



4-1.py

字串變數初始化

```
string01 = "1,2,3,4"
```

```
print(string01)
```

替換字串

```
'''
```

```
string.replace(str1, str2)
```

將 string 中的 str1 替換成 str2

```
'''
```

```
string02 = "Alex"
```

```
string02 = string02.replace('ex', 'len')
```

```
print(string02)
```

去除兩側空格

```
'''
```

```
string.strip()
```

去除字串 string 左、右兩邊的空格

```
'''
```

```
string03 = "          ___ccc___          "
```

```
print(string03)
```

```
print(string03.strip())
```

字串變成小寫或大寫

```
'''
```

```
string.lower()
```

將字串 string 裡的字母全部改成小寫

```
'''
```

```
print("CAR".lower())
```

```
'''
```

```
string.upper()
```

將字串 string 裡的字母全部改成大寫

```
'''
```

```
print("good".upper())
```

補充：字串格式化

- 輸出結果的時候，若是需要嵌入必要的文字，建立字串樣版 (string template)，或是嘗試將文字進行排列與調整，可以將字串格式化。

- 使用 %

語法	說明
%s	插入 字串 的值到字串中
%f	插入 浮點數 的值到字串中
%d	插入 整數 的值到字串中
%%	輸出百分比 (類似跳脫字元)

- 使用 .format()
 - 使用 f-string (Python 版本 > 3.6)

• 使用 %

4-1-1.py

```
'''
格式化字串 - 使用 %
'''

# 多組文字
msg = '%s, %s!' % ('Hello', 'World')
print(msg)

# 整數
msg = 'I am %d years old.' % 5
print(msg)

# 文字與整數
msg = '%s is %d years old.' % ('Alex', 18)
print(msg)

# 指定寬度 (維持 10 個字元長度, 預設向右對齊)
msg = '%10s' % 'Hello'
print(msg)
```

```
# 靠左對齊 (維持 10 個字元長度, 向左對齊)
msg = '[-10s]' % 'Hello'
print(msg)

# 指定浮點數位數
msg = ' [%8.3f]' % 12.3456
print(msg)

# 指定文字長度上限 (只有文字, 才在格式化字串中加入「.」來限定字串長度)
msg = ' [%3s]' % 'Hello'
print(msg)

# 空白補 0
msg = ' [%06.2f]' % 3.1415926
print(msg)
```

• 使用 .format()

4-1-2.py

```
'''
格式化字串 - 使用 string.format()
'''

# 嵌入文字
msg = '{} {}'.format('Hello', 'World')
print(msg)

# 改變參數順序
msg = '{1} {0}'.format('Hello', 'World')
print(msg)

# 指定寬度
msg = '{:10}'.format('Hello')
print(msg)

# 靠右對齊
msg = '{:>10}'.format('Hello')
print(msg)
```

```
# 靠左對齊
msg = '{:<10}'.format('Hello')
print(msg)

# 置中對齊
msg = '{:^10}'.format('Hello')
print(msg)

# 指定浮點數位數
msg = '{:8.3f}'.format(12.3456)
print(msg)

# 指定文字長度上限（只有文字，才在格式化字串中
加入「.」來限定字串長度）
msg = '{:.3}'.format('Hello')
print(msg)

# 空白補 0
msg = '{:06.2f}'.format(3.1415926)
print(msg)
```

• 使用 f-string (Python 版本 > 3.6)

4-1-3.py

```
# f-string (又稱作 formatted string literals,  
version >= 3.6)  
name = "Jarvis"  
age = 18  
result = f"name: {name}, age: {age}"  
print(result)  
  
# 靠左對齊、靠右對齊  
result = f"name: [{name:<10}], age: [{age:>10}]"  
print(result)  
  
# 靠左對齊、靠右對齊，字數不足，補「X」  
# (沒寫補什麼，預設空格)  
result = f"name: [{name:X<10}], age: [{age:X>10}]"  
print(result)
```

```
# 置中對齊  
result = f"name: [{name:^10}], age: [{age:^10}]"  
print(result)  
  
# 指定文字長度上限 (只有文字，才在格式化字串中加入  
# 「.」來限定字串長度)  
result = f"name: [{name:^10.2}]"  
print(result)
```


列表 list 介紹

什麼是 list ?

- list 是一種「串列」資料型態
- 用中括號 [] 包住多個元素
- 有順序、可重複、可修改

範例： [1, 2, 3]

[1 , 2 , 3]

基本語法

```
nums = [10, 20, 30]
```

- ✓ 使用中括號 [] 建立 list
- ✓ 元素以逗號 , 分隔
- ✓ 可以存放任何型別的資料

常見操作



讀取： `nums[0] → 10`



新增： `nums.append(40)`



修改： `nums[1] = 99`



刪除： `nums.pop()`



長度： `len(nums) → 3` (操作前)

程式碼範例

```
nums = [1, 2, 3]
nums.append(4)
nums[0] = 10
print(nums)
```

執行結果

```
[10, 2, 3, 4]
```



重點提醒

- ✓ list 可存放不同型別資料
- ✓ 內容可以修改
- ✓ 常搭配 `append()`、`pop()` 使用



4-2.py

```
# 初始化一個 list
ids = [1, 2, 3, 4, 5, 6]
print(ids)
```

```
# 新增元素在 list 尾端
ids.append(7)
print(ids)
```

```
# 修改索引位置的元素
ids[4] = 99
print(ids)
```

```
# 刪除索引位置的元素
ids.pop(4)
print(ids)
```

```
# 新增元素 9 在指定索引 1，其餘元素往後移
ids.insert(1, 9)
print(ids)
```

```
# 移除指定的值
ids.remove(9)
print(ids)
```

元組 tuple 介紹

什麼是 tuple?

- tuple 是一種「元組」資料型態
- 用小括號 () 包住多個元素
- 有順序、可重複、但不可修改

(1 , 2 , 3)

基本語法

```
point = (10, 20)
```

```
one_item = (5,)
```

- ✓ 使用小括號 ()
- ✓ 元素以逗號分隔
- ✓ 單一元素需加逗號，避免被誤判

常見操作



讀取：point[0] → 10



長度：len(point) → 2



解包：x, y = point



串接：(1, 2) + (3, 4)



重複：("a",) * 3 → ('a', 'a', 'a')

程式碼範例

```
point = (3, 5)
print(point[0])
print(len(point))

x, y = point
print(x, y)
```

執行結果

```
3
5
3 5
```

重點提醒



tuple 建立後不能修改



✓ 單一元素 tuple 要加逗號



✓ 可作為字典的 key (若元素可雜湊)

4-4.py

```
# Tuple 初始化: 第一種
myTuple01 = ("Big", "Data", "is", "Good")
print(myTuple01)

# Tuple 初始化: 第二種
myTuple02 = "AIOT", "is", "Good"
print(myTuple02)

# 透過指定索引輸出值
print(myTuple01[1])
print(myTuple02[1])

# 複數變數修改值
a = 10
b = 20
print("交換前: a = {}, b = {}".format(a, b))
a, b = b, a
print("交換後: a = {}, b = {}".format(a, b))
```

```
# tuple 不可以修改指定索引的值
myTuple = ("Big", "Data", "is", "Good")
myTuple[3] = "Best"
print(myTuple)

# 用 for 迴圈逐一輸出資料
for value in myTuple01:
    print(value)

# 用 len 計算 tuple 資料個數
print(len(myTuple02))
```


字典 dict 介紹

什麼是 dict ?

- dict 是一種「字典」資料型態
- 以 鍵:值 (key:value) 成對儲存資料
- 透過鍵快速取得對應的值

key (鍵) value (值)

name



"Jarvis"

age



18

基本語法

```
student = {  
    "name": "小明",  
    "age": 20,  
    "score": 95  
}
```

- ✓ 大括號 { } 表示 dict
- ✓ 鍵與值用冒號 ; 分隔
- ✓ 每組資料用逗號 , 分開

常見操作



讀取 :

student["name"] → 小明



新增 :

student["city"] = "台北"



修改 :

student["age"] = 21



刪除 :

del student["score"]

程式碼範例

```
student = {"name": "小明", "age": 20}  
print(student["name"])  
student["age"] = 21  
print(student)
```

執行結果

```
小明  
{'name': '小明', 'age': 21}
```



重點提醒

- ✓ 鍵 (key) 通常不可重複
- ✓ 常用字串當作鍵
- ✓ dict 很適合表示一筆資料



4-4.py

```
# 初始化 dict
dict01 = {"蘋果": 100, "橘子": 20, "番茄": 50}
print(dict01)

# 印出蘋果的價格
print(dict01["蘋果"])

# 修改橘子的價格
dict01["橘子"] = 30
print(dict01["橘子"])

# 刪除 番茄
del dict01["番茄"]
print(dict01)
```

```
dict02 = {
    'name': 'Jarvis',
    'age': 18,
    'info': {
        'nickname': 'boatman',
        'favorite_role': 'IronMan',
        'phone_number': [
            '0911111111',
            '0922222222',
            '0933333333'
        ]
    }
}

# 取得 nickname
print( dict02['info']['nickname'] )

# 取得所有 phone_number
for number in dict02['info']['phone_number']:
    print(number)
```

4-4.py

```
# dict 使用 for 迴圈
fruit_dict = {
    "apple": 100,
    "banana": 30,
    "coconut": 50
}

# 1. 取得所有的 key
for key in fruit_dict:
    print(key)

# 2. 取得所有的 value
for value in fruit_dict.values():
    print(value)

# 3. 取得所有的 key-value
for key, value in fruit_dict.items():
    print(key, value)
```

集合 set 介紹

什麼是 set ?

- set 是一種「集合」資料型態
- 元素不重複，會自動去重
- 無順序，不能用索引取值

{ 1 2 3 }

基本語法

```
nums = {1, 2, 3}
empty_set = set()
```

- ✓ 用大括號 { } 建立 set
- ✓ 元素以逗號分隔
- ✓ 空集合需用 `set()` 建立

常見操作

+ 新增: `nums.add(4)`

🗑 刪除: `nums.remove(2)`

🔍 成員測試: `3 in nums` → `True`

U 聯集: `a | b`

∩ 交集: `a & b`

程式碼範例

```
nums = {1, 2, 2, 3}
nums.add(4)
print(nums)
print(3 in nums)
```

執行結果

```
{1, 2, 3, 4}
True
```

重點提醒

💡 set 常用來去除重複資料

- ✓ 不支援索引
- ✓ 可做聯集、交集等集合運算

4-5.py

```
# Set 初始化：第一種
class_set = {"Big", "Data", "is"}
print(class_set)

# 一筆筆讀資料：for 迴圈
for data in class_set:
    print(data)

# 總共有幾筆資料：len()
print(len(class_set))

# 因為沒有序號、索引的概念，所以無法透過指定索引輸出
print(class_set[0])

# 添加一筆資料：add()
class_set.add("the")
class_set.add("best")
```

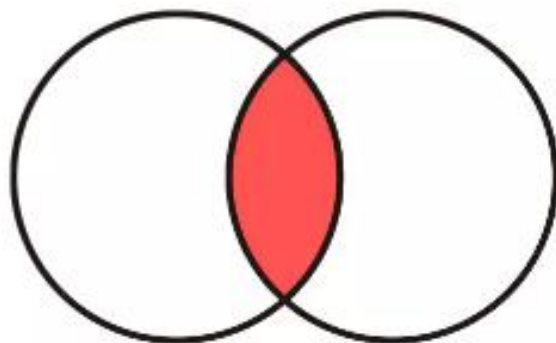
```
# 此時新增重複的資料，不會增加
class_set.add("Big")
print(class_set)

# 添加多筆資料：update()
class_set.update(["AIOT", "AIPE", "BDSE"])
print(class_set)

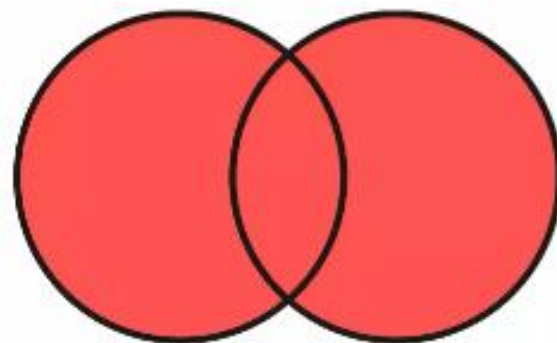
# 刪除元素：discard() or remove()
class_set.discard("is")
```

set 的四種運算型態

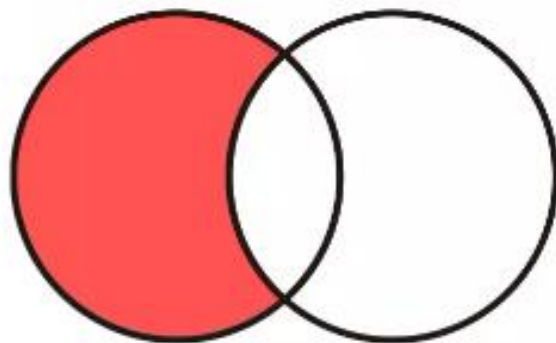
- 交集($A \& B$)
(Intersection)
- 聯集($A | B$)
(Union)
- 差集($A - B$)
(Difference)
- 對稱差集($A \wedge B$)
(Symmetric Difference)



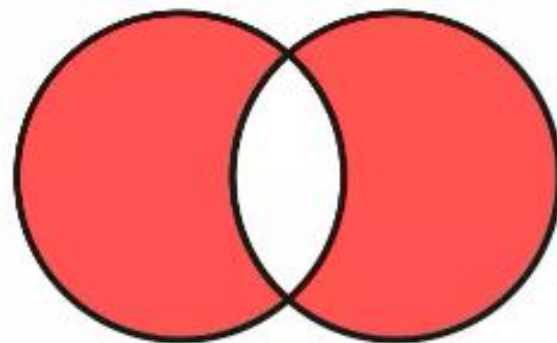
交集 intersection



聯集 union



差集 difference



對稱差集 symmetric_difference

- 序列物件分割 (Slicing)

- 序列物件包含 String、List 及 Tuple等，可透過其順序取得元素
- 中括號為常見物件的索引值取值 (Indexing) 或分割之語法
 - 索引值取值指的是取出一個值
 - 分割指的是取出一部分值

索引值：	0	1	2	3	4	5	6	7	8	9	10	11
	'H'	'e'	'l'	'l'	'o'	','	'W'	'o'	'r'	'l'	'd'	'!'
索引值：	-12	-11	-10	-9	-8	-7	-6	-5	-4	-3	-2	-1

圖：把 sequence 當成 list 來存取

4-6.py

```
# 初始化一個字串
myStr = 'Hello,World!'

'''
透過冒號 (:) 進行分割
'''

# 從頭取到 5 (不包含第 5 個元素，實際為 0 ~ 4)
print( myStr[0:5] ) # 索引從 0 開始，到 5 - 1 的索引位置

# 從頭取到 5 (不包含第 5 個元素，實際為 0 ~ 4)
print( myStr[:5] ) # 冒號前面留空，效果跟 [0:5] 一樣

# 從 7 取到尾
print( myStr[7:] ) # 冒號後面留空

# 從 7 取到 9 (不包含第 9 個元素，實際為 7 ~ 8)
print( myStr[7:9] )
```

```
'''
使用負號
'''

# 從倒數第 10 取到倒數第 7 (不包含倒數第 7 的元素)
print( myStr[-10:-7] )

# 從倒數第 5 取到尾
print( myStr[-5:] )

# 從頭取到倒數第 7 (不包含倒數第 7 的元素)
print( myStr[:-7] )

# 取得檔案全名
string06 = "/home/jarvis/ebook.txt"
list06 = string06.split("/")
print(list06[-1])
```

Module 5.

函式

定義函式

- 函式 (function) 將經常使用的程式碼打包起來，變成一種模組，每執行一次函式，相當於把模組的程式碼全部執行過，透過 **def** 關鍵字來定義。

基本用法

函式的定義方式

def 函式名稱():

陳述句(statement，代表程式要執行的動作)

執行函式，直接使用函式名稱，後面加上()

函式名稱()

• 變數傳遞

回傳值(結果)	傳遞變數
<pre># 傳遞在函式當中執行的結果 def 函式名稱(): return 您的執行結果 # 函式執行完畢後，會將結果回傳給變數 變數 = 函式名稱() # 定義函式 def get_name(): name = "Doraemon" return name # 執行函式 result = get_name() # 輸出回傳結果 print(result)</pre>	<pre># 傳遞在函式當中執行的結果 def 函式名稱(參數[, 參數2,...]): return 整合變數後的結果 # 函式執行完畢後，會將結果回傳給變數 變數 = 函式名稱(變數[, 變數2,...]) # 定義函式 def get_greeting(name): return "Hello, " + name # 執行函式 result = get_greeting("Jarvis") # 輸出回傳結果 print(result)</pre>

區域變數與全域變數

- 當 function 區塊 (block) 內外都有相同的變數名稱：
 - function 區塊內部的變數，叫作**區域變數 (local variable)**。
在函式內部會優先使用**區域變數**。
 - 如果變數在程式碼的最外層，沒有被任何 function 區塊 (block) 包起來，該變數叫作**全域變數 (global variable)**，整個程式都能存取。
- 想在函式內部使用、修改全域變數的值，必須在變數前加上 global 關鍵字。
- 不能在加上 global 的時候進行變數初始化。

- 使用 global 讓函式存取全域變數的值

區域變數無法修改全域變數	區域變數可以修改全域變數
<pre># 全域變數 name = 'Bill' # 定義函式 def 函式名稱(): # 區域變數 name = 'Doraemon' # 執行函式 函式名稱() # 輸出 name print(name) # 輸出 Bill</pre>	<pre># 全域變數 name = 'Bill' # 定義函式 def 函式名稱(): # 使用 global 關鍵字， # 讓函式可以取存全域變數的值 global name # 區域變數 name = 'Doraemon' # 執行函式 函式名稱() # 輸出 name print(name) # 輸出 Doraemon</pre>

匿名函式 Lambda

- 所謂匿名函式 (anonymous function)，許多程式語言都有支援，在程式語言 Python 當中叫作 Lambda，只需要撰寫一行的程式碼，就有一般函式的效果，相對其它函式定義方法來得簡潔。
- 將變數作為函式名稱 = lambda 參數1[,參數2,...]：執行回傳結果

一般函式	Lambda
<pre>def add(x, y): return x * y print(add(3, 5))</pre>	<pre>add = lambda x, y : x * y print(add(3, 5))</pre>
<pre>def abs(x): if x >= 0: return x else: return -x print(abs(5)) print(abs(-5))</pre>	<pre>abs = lambda x : x if x >= 0 else -x print(abs(5)) print(abs(-5))</pre>

Module 6.

物件導向程式設計

基本認識物件導向

- 物件導向程式設計 (Object-Oriented Programming) 是幾乎把所有在 Python 中的東西都看成物件 (object)，擁有它自己的屬性 (attribute) 和方法 (methods)。
- 例如車子有自己的烤漆顏色、4 門或 5 門、車身的寬高、型號等等，這些是屬性；車子可以按下喇叭、煞車、轉彎、加速等，都是它的方法。
- 任何東西都擁有自己的屬性和方法，將這樣的概念用在程式設計上，就叫作物件導向程式設計。

類別 (Class)

- 類別就像是一個樣式的名稱，裡面已經事先定義好基本的、通用的**屬性和方法**，例如人類通常會有兩隻手、兩條腿 (**attribute**)，同時也會爬行、走路、跑、跳 (**function**)等。

屬性

- **類別屬性 (class attribute)** 是屬於類別本身的變數，而不屬於類別的任何特定實例。這意味著所有該類別的**實例**都共享同一個類別屬性的值。

方法 (Methods)

- 方法就是定義在 class 內的函式 (functions) ，都是執行一連串在程式區塊中定義好的程式碼 (code) ，在每一個類別 (class) 中，都會有一些 Python 內建且需要在一開始定義的函式。
- 在建立類別的時候，常常會看到 `__init__(self)` ，指的是建立物件 (或稱為實體) 的時候，預設第一時間會執行的方法，稱為建構子 (constructor) 或 建構函式。

class 使用方式

```
class 類別名稱:
```

```
    def __init__(self):
```

```
        print('Python')
```

```
    def 方法1(self):
```

```
        return 'Hello World!'
```

```
    def 方法2(self):
```

```
        return 'Good job!'
```

```
物件(實體) = 類別名稱() # 建立物件(又稱實體化)時，直接輸出 Python
```

```
print( 物件.方法1() ) # 輸出 Hello World!
```

```
print( 物件.方法2() ) # 輸出 Good job!
```

- 在建構函式內有設定參數，代表在建立該物件時一定要帶參數

```
class 類別名稱:
```

```
    def __init__(self, 參數1, 參數2):
```

```
        self.實例屬性1 = 參數1
```

```
        self.實例屬性2 = 參數2
```

```
        self.實例屬性3 = 自訂值
```

```
物件(實體) = 類別名稱('Hello', 'World')
```

```
print(物件.實例屬性1) # 輸出 Hello
```

```
print(物件.實例屬性2) # 輸出 World
```

```
print(物件.實例屬性3) # 輸出 在程式中的自訂值
```



```
class Car:    # 定義一個 類別 ( class ) , 叫做 Car

    wheels = 4    # 定義 Car 的 類別屬性 ( class Attribute ) , wheels

    def __init__(self, brand, model):

        self.brand = brand

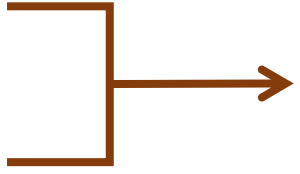
        self.model = model

        self.doors = 4

    def display_info(self):

        return f"The {self.brand} {self.model} has {Car.wheels} wheels."
```

```
class Car:
    wheels = 4
```



使用 Car 的 類別屬性
Car.wheels # 得到 4

```
def __init__(self, brand, model):
```

```
    self.brand = brand
```

```
    self.model = model
```

```
    self.doors = 4
```

```
def display_info(self):
```

```
    return f"The {self.brand} {self.model} has {Car.wheels} wheels."
```

```
class Car:
```

```
    wheels = 4
```

建構函式 類別的實例 建構函式的參數

```
def __init__(self, brand, model):
```

```
    self.brand = brand
```

```
    self.model = model
```

```
    self.doors = 4
```

```
def display_info(self):
```

```
    return f"The {self.brand} {self.model} has {Car.wheels} wheels."
```

```
class Car:
```

```
    wheels = 4
```

```
    def __init__(self, brand, model):
```

```
        self.brand = brand
```

```
        self.model = model
```

```
        self.doors = 4
```

```
    def display_info(self):
```

```
        return f"The {self.brand} {self.model} has {Car.wheels} wheels."
```

實例屬性



```
class Car:
```

```
    wheels = 4
```

```
    def __init__(self, brand, model):
```

```
        self.brand = brand
```

```
        self.model = model
```

```
        self.doors = 4
```

```
    def display_info(self):
```

```
        return f"The {self.brand} {self.model} has {Car.wheels} wheels."
```

實例屬性

用 參數 設定 實例屬性的值

```
class Car:
```

```
wheels = 4
```

創建 Car 類別的一個實例

```
car1 = Car("Toyota", "Camry")
```

使用 Car 的實例屬性

```
car1.brand # 得到 Toyota
```

```
def __init__(self, brand, model):
```

```
    self.brand = brand
```

```
    self.model = model
```

```
    self.doors = 4
```

```
def display_info(self):
```

```
    return f"The {self.brand} {self.model} has {Car.wheels} wheels."
```

```
class Car:
```

```
wheels = 4
```

創建 Car 類別的一個實例

```
car1 = Car("Toyota", "Camry")
```

```
def __init__(self, brand, model):
```

```
    self.brand = brand
```

```
    self.model = model
```

```
    self.doors = 4
```

```
def display_info(self): # 實例的方法(method)
```

```
    return f"The {self.brand} {self.model} has {Car.wheels} wheels."
```

使用實例的屬性

使用類別的屬性

使用 car1 物件的方法

car1.display_info()

繼承 (inheritance)

- 繼承 (Inheritance) 是建立一個新的類別 (class) 時，把前一個定義好的類別(父類別)內容拿過來使用，包括屬性和方法，同時也可以在新類別當中定義新的屬性和方法。
- 繼承的好處：
 - 代碼重用：可以使用父類的方法和屬性，無需重新編寫。
 - 擴展性：可以在不修改現有類別的情況下添加新功能。
 - 可維護性：修改父類的程式碼，所有子類都會獲得更新。

inheritance

```
class Vehicle:
    def info(self):
        return f"This is class : {self.__class__.__name__}"

print(Vehicle().info())

# class Car 繼承上面定義好的 class Vehicle
class Car(Vehicle):
    pass

# 可以使用 info() 這個 method
print(Car().info())
```

覆寫 (Override)

- 在 Python 中，覆寫父類別的方法非常直接。當子類別中有一個與父類別**同名的方法**時，允許子類別改變或擴展從父類別繼承的行為，這個方法就自動覆寫了父類別中的同名方法。



當我們在子類別 Override 某個方法，但又想保留父類別的原始方法時，可以使用 **super()** 來調用繼承的父類別方法

Override

父類別

```
class Vehicle:

    def __init__(self, brand, model):

        self.brand = brand

        self.model = model

    def display_info(self):

        return f"Brand: {self.brand}, Model: {self.model}"
```

Override

```
class Car(Vehicle):  
    def __init__(self, brand, model, car_type):  
        super().__init__(brand, model) # 在子類別中調用父類別的方法  
        self.car_type = car_type  
  
    def display_car_info(self):  
        return f"{self.display_info()}, Car Type: {self.car_type}"
```

Module 7.

例外處理

認識例外處理

- 例外 (Exception) 就是程式執行過程中，發生錯誤，影響程式的正常執行，所立刻執行的事件 (event)。
- 通常程式執行都會被中斷，偶爾也會顯示一些造成問題的原因；也可以自訂例外發生時的程式碼，不見得一定要使用預設的警示結果。

常見的事件類別

事件類別	說明
BaseException	基本例外事件類別
Exception	最常用的例外事件類別
IOError	輸入/輸出發生錯誤的事件類別
NameError	變數尚未宣告所發生的事件類別
IndexError	序列中沒有這個索引(index)的事件類別
RuntimeError	執行過程中發生錯誤的事件類別
SyntaxError	Python 語法錯誤的事件類別
KeyboardInterrupt	使用 Ctrl + C 取消程式執行的事件類別
IndentationError	縮排錯誤的事件類別
Warning	顯示警告訊息的事件類別 (通常不會中斷程式)
DeprecationWarning	功能、指令、語法等不再被支援的事件類別
RuntimeWarning	可能造成程式執行錯誤的事件類別
SyntaxWarning	程式碼語法可能有問題的事件類別

try-except 陳述

- 例外的處理，可以使用「try: ... except: ...」語法，try: 是預設的程式區塊，當 try: 程式區塊的程式碼在執行期間發生錯誤，會直接中斷，跳到 except: 程式區塊來繼續執行。

try:

預設正常執行程式的程式區塊

except 例外事件類別名稱1:

拋出例外時，符合例外事件類別名稱1，則在這個程式區塊繼續執行

except 例外事件類別名稱2:

拋出例外時，符合例外事件類別名稱2，則在這個程式區塊繼續執行

finally:

無論是否正常執行，最後都會執行這裡的程式區塊

7-1.py

```
# 可以連續定義數個例外處理機制，  
# 最符合的例外事件類別會先拋出  
try:  
    print(x)  
except NameError:  
    print("x 尚未宣告")  
except:  
    print("某個東西出錯了")  
  
# 多層例外處理：  
try:  
    print(x)  
    try:  
        print(y)  
    except NameError as err:  
        print("NameError:")  
        print(err)  
except Exception as err:  
    print("Exception:")  
    print(err)
```

例外處理 raise 陳述

- 如果有自訂例外的需求，可以使用 raise 語法來拋出例外訊息。

7-2.py

```
# 自訂例外處理
x = -1
if x < 0:
    raise Exception("數字小於 0")

# 自訂型別錯誤的例外處理
x = "hello"
if type(x) is not int:
    raise TypeError("只接受整數型別")
```

Module 8.

檔案處理

open() 函式

- 可以使用內建的 open() 函式來進行檔案處理，其中包含了 4 種檔案模式：
 - "r": Read (讀取檔案)
 - "a": Append (附加資料到檔案內容的最尾端或最後一行)
 - "w": Write (寫入檔案)
 - "x": Create (建立檔案)
 - "t": Text (文字模式)
 - "b": Binary (二進制模式，例如用於 images 的讀取)

讀取檔案

8-1.py

讀取檔案內容 - 現在常用的寫法（執行完畢會自動關閉檔案）

```
with open(file="./2-1.py", mode="r", encoding="utf-8") as f:  
    print(f.read())
```

```
print("=" * 50) # 印出 50 個 等號
```

一行一行讀出來

```
with open(file="./2-1.py", mode="r", encoding="utf-8") as f:  
    for line in f:  
        print(line) # 每一行後面會用 \n 結尾，所以輸出會有多次斷行的效果
```

寫入檔案

8-2.py

```
# 文字內容
```

```
text = '''
```

```
我這一生如履薄冰，你說我能走到對岸嗎
```

```
'''
```

```
# 寫入檔案
```

```
with open("./8-2.txt", "w", encoding="utf-8") as f:
```

```
    f.write(text + "\n")
```

```
# 寫入最後一行（附加文字在檔案內容最後一行）
```

```
with open("./8-2.txt", "a", encoding="utf-8") as f:
```

```
    f.write("哟，这不薄冰哥嗎，今天還沒走到對岸呢" + "\n")
```

刪除檔案

8-2.py

```
# 匯入套件
import os

# 檔案路徑
file_path = './8-2.txt'

# 如果指定路徑的檔案存在，則進行刪除
if os.path.exists(file_path):
    os.remove(file_path)
else:
    print('Not found')
```

JSON 格式

在JSON未普遍之前，工程師們如果要交換資料，通常會寫在Excel裡 - 轉成 (XML)，或是把資料寫在文字檔裡 (TXT) 等等其他交換資料方式，處理的都是一個個的檔案，開檔、讀檔、更檔、寫檔，總是沒有那麼方便，所以這也是JSON檔，一下出來就變主流的主要原因。

JSON (JavaScript Object Notation) 全名是 JavaScript 物件表示法，是一種輕量級的**資料交換格式**。它使用純文字來記錄和傳輸結構化的資料，特色是**人類好讀、機器好懂**，目前被廣泛用在網頁與伺服器之間的資料傳遞。

JSON 使用方式

函式	用途	資料方向
<code>json.loads()</code>	解析 JSON 字串	JSON 字串 → Python 物件
<code>json.dumps()</code>	產生 JSON 字串	Python 物件 → JSON 字串
<code>json.load()</code>	讀取 JSON 檔案	JSON 檔案 → Python 物件
<code>json.dump()</code>	寫入 JSON 檔案	Python 物件 → JSON 檔案

Python 與 JSON 型別對照

Python	JSON
<code>dict</code>	<code>object</code>
<code>list</code> 、 <code>tuple</code>	<code>array</code>
<code>str</code>	<code>string</code>
<code>int</code> 、 <code>float</code>	<code>number</code>
<code>True</code>	<code>true</code>
<code>False</code>	<code>false</code>
<code>None</code>	<code>null</code>

Module 9.

模組

定義模組

- 將常用的程式碼 (通常是函式) 整合在一個 .py 檔案中，變成一個函式庫 (library)，或是重複使用的工具，在其它程式需要用到時候，可以不用重新撰寫，只要透過 import 語法將其引入、匯入，即可在當前的程式檔案裡使用。

myModule.py

計算幾次方

```
def pow(x, y):  
    return x**y
```

簡單斷句

```
def segment_sentence(text):  
    list_sentences = text.split(' ')  
    return list_sentences
```

使用模組-1

- 匯入模組的程式檔，通常是主程式；myModule.py 就是自訂模組，被主程式匯入。

9-1.py

```
import myModule
# 計算幾次方
num = myModule.pow(2, 3)
print(num)

# 簡單斷句
txt = "I will always love you"
list_result = myModule.segment_sentence(txt)
print(list_result)
```

使用模組-2

- 直接從模組匯入特定函式，這一種較為常用，明確的表示從模組引用了哪些函式。

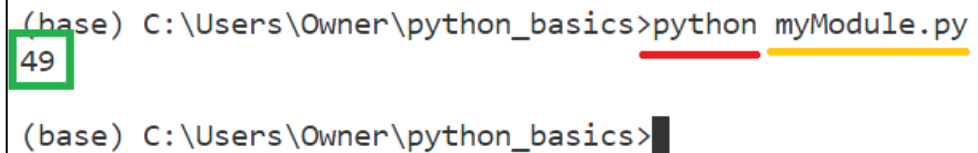
9-2.py

```
# 直接從模組中匯入函式
from myModule import pow, segment_sentence

# 計算幾次方
num = pow(2, 3)
print(num)

# 簡單斷句
txt = "I will always love you"
list_result = segment_sentence(txt)
print(list_result)
```

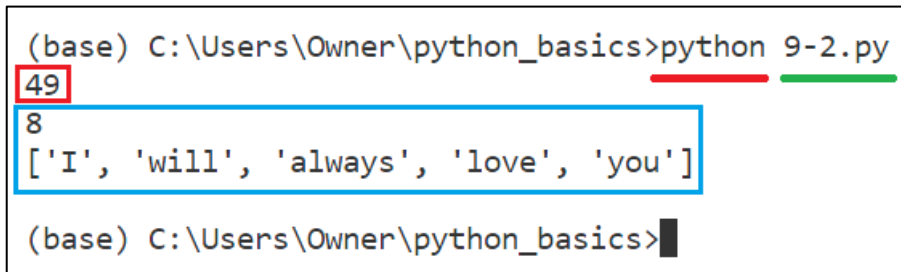
- 如果想要測試自訂模組的執行結果是否如預期，
可以執行 myModule.py：



```
(base) C:\Users\Owner\python_basics>python myModule.py
49
(base) C:\Users\Owner\python_basics>
```

圖：myModule.py 執行結果

- 此時執行 9-2.py 的話，會發生什麼事？



```
(base) C:\Users\Owner\python_basics>python 9-2.py
8
['I', 'will', 'always', 'love', 'you']
(base) C:\Users\Owner\python_basics>
```

圖：突然輸出 49 這個模組內部測試結果

- 那是因為匯入模組的時候，**Python 會將模組內部的程式碼 (myModule.py) 由上而下執行過一遍**，連同模組的執行輸出也顯示在主要程式 (9-2.py) 裡。若是希望模組測試的結果，不要在主程式執行階段輸出，此時要將 myModule.py 改成：

myModule.py

```
'''
```

```
將模組當作主程式來執行，
```

```
此時 myModule.py 的 __name__ 就會等於 __main__，
```

```
__main__ 就是主程式的意思
```

```
'''
```

```
if __name__ == "__main__":
```

```
    print( pow(7, 2) )
```

__name__ 內建變數的用法

- __name__ 是內建變數，用來確認匯入模組的名稱

例如：myModule (會自動去掉 .py)

如果執行的主程式檔案裡面有輸出 __name__ 它的值就會是 __main__。



雖然主程式 (9-2.py) 執行時，模組的輸出不會顯示出來，實務上通常都會在撰寫程式碼的檔案加上「`if __name__ == "__main__":`」來確保每個程式進行模組化 (modularization) 時，能避免掉不必要的麻煩

9-2.py

直接從模組中匯入函式

```
from myModule import pow, segment_sentence
```

```
if __name__ == "__main__":
```

```
    # 計算幾次方
```

```
    num = pow(2, 3)
```

```
    print(num)
```

```
    # 簡單斷句
```

```
    txt = "I will always love you"
```

```
    list_result = segment_sentence(txt)
```

```
    print(list_result)
```

Module 10.

裝飾器

Decorator (裝飾器)

- 裝飾器是一種Python的高級功能，它允許你**修改或擴展函式**或方法的行為，而**無需直接修改它們的程式碼**。
- 裝飾器本身是一個函式，它接受一個函式作為參數並返回一個新的函式。通常，裝飾器的名稱以 **@** 符號開頭，並放在要裝飾的函式定義之前。

初建立 decorator

```
# 定義一個裝飾器函數

def my_decorator(func):

    def wrapper():

        print("在函式執行之前做一些事情")

        func() # 呼叫原始函式

        print("在函式執行之後做一些事情")

    return wrapper
```

```
# 使用裝飾器

@my_decorator

def say_hello():

    print("Hello decorator ! ")

# 調用裝飾後的函式

say_hello()
```

```
def my_decorator(func):  
  
    def wrapper():  
  
        print("在函式執行之前做一些事情")  
  
        func() # 執行 say_hello()  
  
        print("在函式執行之後做一些事情")  
  
    return wrapper
```

```
@my_decorator  
  
def say_hello():  
  
    print("Hello decorator ! ")  
  
say_hello()
```



當我們調用 `say_hello()` 時，它實際上執行的是 `wrapper` 函式，這允許我們在函式執行前後添加自己的邏輯。

*args 和 **kwargs

- ***args** 和 ****kwargs** 是Python中的特殊參數，它們允許函式接受不定數量的位置參數和關鍵字參數。
- ***args** 表示接受任意數量的位置參數，****kwargs** 表示接受任意數量的關鍵字參數。在這邊使用可以確保裝飾器函式應用於各種不同的函式，而不必關心它們的參數。

PyMySQL

Chapter1

PyMySQL 介紹

Python 連線資料庫

1. PyMySQL (較為簡單，容易上手)
2. MySQL Connector/Python (MySQL 官方提供)
3. SQLAlchemy (ORM 框架，可搭配 PyMySQL / mysqlclient)
4. asyncmy (非同步 PyMySQL 替代品)
5. mysqlclient (原為 MySQLdb，需安裝 C 編譯器)

PyMySQL – 建立連線

```
import pymysql
# 建立 MySQL 資料庫連線
connection = pymysql.connect( # connect 函式回傳 Connection 物件
    host='localhost', # MySQL 伺服器的 IP 位址
    user='root',        # 登入 MySQL 的使用者名稱
    password='p@ssw0rd', # 登入 MySQL 的密碼
    port=3306,          # MySQL 伺服器的端口
)
```

RuntimeError: 'cryptography' package is required for sha256_password or caching_sha2_password auth methods
Output is truncated. View as a [scrollable element](#) or open in a [text editor](#). Adjust cell output [settings...](#)

註：如果出現該錯誤，需執行 `pip install cryptography`，並重啟 notebook

PyMySQL – cursor 常用的方法和屬性

方法/屬性	說明
<code>execute(query, args=None)</code>	執行單一 SQL 指令，支援參數化查詢以避免 SQL injection。
<code>executemany(query, args)</code>	一次執行多筆相同格式的 SQL 指令，常用於批次插入或更新。
<code>fetchone()</code>	取得查詢結果的下一筆資料 (tuple)，若無資料則回傳 None。
<code>fetchall()</code>	取得所有查詢結果資料 (list of tuples)，若無資料則回傳空 list。
<code>fetchmany(size)</code>	取得指定數量的資料列，回傳 list of tuples。
<code>close()</code>	關閉 cursor，釋放資源。
<code>rowcount</code> (屬性)	回傳最後一次執行 SQL 所影響的資料列數量 (如 UPDATE, DELETE)。
<code>lastrowid</code> (屬性)	回傳最後一次 INSERT 操作所自動產生的 AUTO_INCREMENT 主鍵值。
<code>description</code> (屬性)	回傳欄位描述資訊 (tuple of column info)，可用來動態取得欄位名稱。

PyMySQL – 查詢資料庫

```
# 建立游標
cursor = connection.cursor()
# 使用游標執行 SQL 語句
cursor.execute("SHOW DATABASES;")
# 取得所有資料
dbs = cursor.fetchall()

print(dbs)

# 關閉游標
cursor.close()
```

PyMySQL – 建議的寫法

```
# 建立游標
with connection.cursor() as cursor:
    query = "SHOW DATABASES;"
    # 使用游標執行 SQL 語句
    cursor.execute(query)
    # 取得所有資料
    databases = cursor.fetchall()
```



離開 `with` 語句縮排時，自動關閉及釋放游標

PyMySQL – 查詢資料庫有哪些資料表

```
with connection.cursor() as cursor:  
    # 使用游標執行 SQL 語句  
    cursor.execute("SHOW TABLES FROM database_name;")  
    # 取得所有資料  
    tables = cursor.fetchall()  
  
print(tables)
```

PyMySQL – 查詢資料

```
with connection.cursor() as cursor:  
    # 使用 DB > TABLE 查詢  
    sql = "SELECT * FROM database_name.table;"  
    cursor.execute(sql)  
    users = cursor.fetchall()  
  
print(users)
```

PyMySQL – 建立連線(指定資料庫)

```
import pymysql
connection = pymysql.connect(
    host='localhost',
    user='root',
    password='p@ssw0rd',
    port=3306,
    database='database_name' # 指定資料庫
)
```

PyMySQL – 查詢資料

```
with connection.cursor() as cursor:  
    sql = "SELECT * FROM table;"  
    cursor.execute(sql)  
    users = cursor.fetchall()  
print(users)
```

```
((1,  
  'Admin',  
  None,  
  'AdminUser',  
  'BioInf/IT',  
  datetime.datetime(2025, 2, 8, 9, 51, 8, 434477)),  
(2,  
  'jarvis_lu',  
  '',  
  'Jarvis Lu',  
  'BioInf/IT',  
  datetime.datetime(2025, 2, 19, 15, 43, 0, 610623)))
```



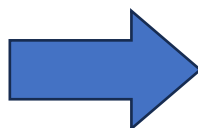
預設回傳的資料型別是 **tuple**
且沒有回傳欄位名稱
不方便後續處理使用

PyMySQL – 建立連線(修改回傳格式)

```
import pymysql
connection = pymysql.connect(
    host='localhost',
    user='root',
    password='p@ssw0rd',
    port=3306,
    database='database_name',
    cursorclass=pymysql.cursors.DictCursor # 查詢結果以字典形式返回
)
```

PyMySQL – 使用 DictCursor 後

```
((1,
  'Admin',
  None,
  'AdminUser',
  'BioInf/IT',
  datetime.datetime(2025, 2, 8, 9, 51, 8, 434477)),
(2,
  'jarvis_lu',
  '',
  'Jarvis Lu',
  'BioInf/IT',
  datetime.datetime(2025, 2, 19, 15, 43, 0, 610623)),
(3, 'Gene', None, 'Gene_Manager', 'BioInf/IT', None),
(4,
  'Jason',
  None,
  'Jason',
  'BioInf/IT',
  datetime.datetime(2025, 2, 19, 15, 48, 39, 29619)))
```



```
[{'ch_name': None,
  'department': 'BioInf/IT',
  'id': 1,
  'last_login': datetime.datetime(2025, 2, 8, 9, 51, 8, 434477),
  'name': 'AdminUser',
  'username': 'Admin'},
{'ch_name': '',
  'department': 'BioInf/IT',
  'id': 2,
  'last_login': datetime.datetime(2025, 2, 19, 15, 43, 0, 610623),
  'name': 'Jarvis Lu',
  'username': 'jarvis_lu'},
{'ch_name': None,
  'department': 'BioInf/IT',
  'id': 3,
  'last_login': None,
  'name': 'Gene_Manager',
  'username': 'Gene'},
{'ch_name': None,
  'department': 'BioInf/IT',
  'id': 4,
  'last_login': datetime.datetime(2025, 2, 19, 15, 48, 39, 29619),
  'name': 'Jason',
  'username': 'Jason'}]
```

configparser – 將帳號密碼寫在設定檔中



如果使用 **GitHub** 管理專案，不要將正式的帳號密碼寫在程式中。

```
config.ini  
[DB]  
host=localhost  
user=user  
password=password  
port=port
```

configparser – 讀取設定檔

```
from configparser import ConfigParser

# 讀取 config.ini 檔案取得資料庫連線資訊
config = ConfigParser()
config.read('config.ini')

print(config['DB']) # 讀取 DB 區塊
print(config['DB']['host']) # 讀取 host
print(config['DB']['user']) # 讀取 user
```

configparser – 從設定檔建立資料庫連線

```
# 建立資料庫連線
```

```
connection = pymysql.connect(  
    host=config.get('DB', 'host'),  
    user=config.get('DB', 'user'),  
    password=config.get('DB', 'password'),  
    port=config.getint('DB', 'port'),  
    cursorclass=pymysql.cursors.DictCursor,  
)
```

PyMySQL – 將 SQL 查詢寫成函式

```
def sql_query(query):  
    connection = pymysql.connect(  
        host=config.get('DB', 'host'),  
        user=config.get('DB', 'user'),  
        password=config.get('DB', 'password'),  
        port=config.getint('DB', 'port'),  
        cursorclass=pymysql.cursors.DictCursor,  
    )  
    with connection.cursor() as cursor:  
        cursor.execute(query) # 將 SQL 語句設為參數  
        result = cursor.fetchall()  
    return result
```

PyMySQL

Chapter2

INSERT / UPDATE

PyMySQL – 建立資料庫

```
with connection.cursor() as cursor:
    sql = """
        CREATE DATABASE IF NOT EXISTS chapter2;
    """

    # 執行建立的 SQL 語句
    cursor.execute(sql)

    # 執行查看資料庫
    cursor.execute("SHOW DATABASES;")

    result = cursor.fetchall()

print(result)
```


PyMySQL – 建立資料表

```
with connection.cursor() as cursor:
    sql = """
        CREATE TABLE IF NOT EXISTS users (
            id INT AUTO_INCREMENT PRIMARY KEY,
            name VARCHAR(255) NOT NULL,
            age INT NOT NULL,
            username VARCHAR(255) NOT NULL,
            password VARCHAR(255) NOT NULL
        );
    """
    cursor.execute(sql)
```

PyMySQL – 寫入資料

```
with connection.cursor() as cursor:
    sql = """
        INSERT INTO users (name, age, username, password)
        VALUES ("Jarvis", 25, "jarvis", "123456")
    """

    # 執行寫入的 SQL 語句
    cursor.execute(sql)

    # 執行查詢資料表
    cursor.execute("SELECT * FROM users;")

    # 取得查詢的所有資料
    result = cursor.fetchall()
```

PyMySQL – `connection.commit()`

如果沒有執行 `commit()`，資料只會存在記憶體，關閉程式或連線後就會消失，不會寫進資料庫。所以只要是會改變資料表內容的操作，都需要在操作完成後呼叫一次 `commit()` 以提交變更。

例如：

- `INSERT` (新增資料)
- `UPDATE` (更新資料)
- `DELETE` (刪除資料)

PyMySQL – 寫入資料

```
with connection.cursor() as cursor:
    sql = """
        INSERT INTO users (name, age, username, password)
        VALUES ("Jarvis", 25, "jarvis", "123456")
    """

    cursor.execute(sql)
    # 提交資料庫的變更
    connection.commit()
    cursor.execute("SELECT * FROM users;")
    result = cursor.fetchall()
pprint(result)
```

PyMySQL – 更新資料

```
with connection.cursor() as cursor:
    sql = """
        UPDATE users SET age = 26, name = "Jarvis Lu" WHERE id = 1;
    """

    cursor.execute(sql)
    # 提交資料庫的變更
    connection.commit()
    cursor.execute("SELECT * FROM users;")
    result = cursor.fetchall()
pprint(result)
```

PyMySQL – 使用格式化字串

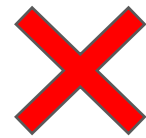
```
with connection.cursor() as cursor:  
    query = """  
        SELECT * FROM main_user LIMIT %s  
    """  
    cursor.execute(query, (1,))  
    result = cursor.fetchall()
```



SQL 語句參數僅支援 %s 佔位符

PyMySQL – SQL injection

```
username = "' OR 1=1; -- "  
password = ""  
with connection.cursor() as cursor:  
    sql = f"""  
    SELECT * FROM users WHERE username = '{username}' AND  
    password = '{password}'  
    """  
  
    cursor.execute(sql)  
    result = cursor.fetchall()  
print(result)
```



勿使用 f-string 搭配 SQL 語句

PyMySQL – SQL injection

```
sql = f"""
SELECT * FROM users WHERE username = '{username}' AND
password = '{password}'
"""
```

如果 username 輸入 ' OR 1=1; -- ，使用 f-string 拼接會變成

```
SELECT * FROM users WHERE username = '' OR 1=1; -- ' AND password = ''
```

導致整個密碼條件失效。

而使用 %s 時，表示一個字串格式化占位符

會將整個參數視為一個完整的字串，無法從中終結SQL指令。

PyMySQL – 寫成函式

```
def insert_user(name, age, username, password):  
    connection = pymysql.connect( ..... )  
  
    with connection.cursor() as cursor:  
        sql = """  
        INSERT INTO user (name, age, username, password)  
        VALUES (%s, %s, %s, %s)  
        """  
  
        cursor.execute(sql, (name, age, username, password))  
        connection.commit()
```

PyMySQL – executemany()

```
users_list = []  
for i in range(101, 201):  
    users_list.append((f'user_{i}', i + 20, f'user_{i}', f'password_{i}'))  
    # [('user_101', 121, 'user_101', 'password_101'), ('user_102', 122, 'user_102', 'password_102'), ...]  
  
with connection.cursor() as cursor:  
    sql = """  
    INSERT INTO user (name, age, username, password)  
    VALUES (%s, %s, %s, %s);  
    """  
  
    cursor.executemany(sql, users_list)  
    connection.commit()
```

PyMySQL

Chapter3

datetime 使用

建立含時間資訊的資料表

```
with connection.cursor() as cursor:
    sql = """
        CREATE TABLE IF NOT EXISTS users (
            id INT AUTO_INCREMENT PRIMARY KEY,
            name VARCHAR(255) NOT NULL,
            age INT NOT NULL,
            username VARCHAR(255) NOT NULL,
            password VARCHAR(255) NOT NULL,
            last_login TIMESTAMP DEFAULT CURRENT_TIMESTAMP
        );
    """

    cursor.execute(sql)
```

寫入資料並查詢

```
with connection.cursor() as cursor:
    sql = """
        INSERT INTO users (name, age, username, password)
        VALUES ("Jarvis", 25, "jarvis", "123456");
    """

    cursor.execute(sql)
    connection.commit()
    cursor.execute("SELECT * FROM users;")
    result = cursor.fetchall()
pprint(result)
```

PyMySQL – datetime

在使用 `cursor.fetchall()` 查詢 SQL 資料庫時，如果資料表中有 `DATETIME` 或 `TIMESTAMP` 類型的欄位，**PyMySQL** 會把這些欄位的值轉成 Python 的 `datetime.datetime` 物件。

```
[{'age': 25,  
  'id': 1,  
  'last_login': datetime.datetime(2025, 5, 25, 15, 52, 22),  
  'name': 'Jarvis',  
  'password': '123456',  
  'username': 'jarvis'}]
```

datetime – 常用的方法

方法/屬性	說明	範例
<code>datetime.now()</code>	取得當前日期與時間	<code>datetime.now()</code>
<code>date.today()</code>	取得當前日期 (無時間)	<code>date.today()</code>
<code>datetime(year, month, day, hour, min, sec)</code>	建立 datetime 物件	<code>datetime(2025, 5, 19, 14, 30)</code>
<code>dt.date()</code>	取得日期部分	<code>datetime.now().date()</code>
<code>dt.time()</code>	取得時間部分	<code>datetime.now().time()</code>
<code>dt + timedelta(days=1)</code>	日期加一天	<code>datetime.now() + timedelta(days=1)</code>
<code>dt1 > dt2</code>	比較兩個 datetime	<code>datetime(2025,1,1) > datetime(2024,1,1)</code>
<code>dt.strftime('%Y-%m-%d')</code>	轉為字串格式	<code>datetime.now().strftime('%Y/%m/%d')</code>
<code>datetime.strptime(str, fmt)</code>	字串轉 datetime	<code>datetime.strptime('2025-05-19', '%Y-%m-%d')</code>
<code>dt.weekday()</code>	傳回 0~6 (週一 ~ 週日)	<code>datetime.now().weekday()</code>

datetime – 格式符號

格式符號	說明	範例
%Y	年（4 位數）	2025
%y	年（2 位數）	25
%m	月（01–12）	05
%d	日（01–31）	19
%H	小時（00–23）	14
%M	分（00–59）	30
%S	秒（00–59）	45
%A	星期幾（英文全名）	Monday
%a	星期幾（縮寫）	Mon
%p	AM / PM	AM

datetime – 使用範例

```
from datetime import datetime

# 取得當前時間
print(datetime.now()) # 2025-05-19 16:11:15.427674

# 取得指定時間的 datetime 物件
print(datetime(2025, 5, 28, 16, 11, 15)) # 2025-05-28 16:11:15

# 將 datetime 輸出成指定格式字串
print(datetime.now().strftime('%Y-%m-%d %H:%M:%S')) # 2025-05-19 16:11:15

# 取得民國年
now = datetime.now()
print(f"民國{now.year - 1911}年{now.month}月{now.day}日") # 114年5月19日
```

PyMySQL – 寫入資料並查詢

```
with connection.cursor() as cursor:
    sql = """
        INSERT INTO users (name, age, username, password)
        VALUES ("Jarvis", 25, "jarvis", "123456");
    """

    cursor.execute(sql)
    connection.commit()
    cursor.execute("SELECT * FROM users;")
    result = cursor.fetchall()
pprint(result)
```

PyMySQL – 寫入datetime資料

```
datetime_now = datetime.now()
with connection.cursor() as cursor:
    sql = """
        INSERT INTO users (name, age, username, password, start_date, last_login)
        VALUES ("Jarvis", 25, "jarvis", "123456", %s, %s);
    """
    cursor.execute(sql, ("2025-05-24", datetime_now))
    connection.commit()
```



可使用

1. 標準 SQL 日期格式 'YYYY-MM-DD HH:MM:SS'
2. datetime 物件

PyMySQL – 更新datetime資料

```
from datetime import datetime, timedelta
yesterday = datetime.now() - timedelta(days=1)
with connection.cursor() as cursor:
    sql = """
        UPDATE users2 SET start_date = %s, last_login = %s
        WHERE id = %s;
    """
    cursor.execute(sql, (yesterday, "2025-05-28 10:30:50", 1))
    connection.commit()
    cursor.execute("SELECT * FROM users2;")
    result = cursor.fetchall()
```

PyMySQL

Chapter4

實例練習

零售超市的銷售記錄

A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P
Row ID	Order ID	Order Date	Ship Date	Ship Mode	Customer ID	Customer Name	Segment	Product ID	Category	Sub-Category	Product Name	Sales	Quantity	Discount	Profit
1	CA-2016-1	11/8/2016	11/11/2016	Second Cla	CG-12520	Claire Gute	Consumer	FUR-BO-10	Furniture	Bookcases	Bush Somerset	261.96	2	0	41.9136
2	CA-2016-1	11/8/2016	11/11/2016	Second Cla	CG-12520	Claire Gute	Consumer	FUR-CH-10	Furniture	Chairs	Hon Deluxe Fa	731.94	3	0	219.582
3	CA-2016-1	6/12/2016	6/16/2016	Second Cla	DV-13045	Darrin Van Huff	Corporate	OFF-LA-10	Office Sup	Labels	Self-Adhesive	14.62	2	0	6.8714
4	US-2015-1	10/11/2015	10/18/2015	Standard C	SO-20335	Sean O'Donnell	Consumer	FUR-TA-10	Furniture	Tables	Bretford CR450	957.5775	5	0.45	-383.031
5	US-2015-1	10/11/2015	10/18/2015	Standard C	SO-20335	Sean O'Donnell	Consumer	OFF-ST-10	Office Sup	Storage	Eldon Fold 'N	22.368	2	0.2	2.5164
6	CA-2014-1	6/9/2014	6/14/2014	Standard C	BH-11710	Brosina Hoffmar	Consumer	FUR-FU-10	Furniture	Furnishings	Eldon Expressi	48.86	7	0	14.1694
7	CA-2014-1	6/9/2014	6/14/2014	Standard C	BH-11710	Brosina Hoffmar	Consumer	OFF-AR-10	Office Sup	Art	Newell 322	7.28	4	0	1.9656
8	CA-2014-1	6/9/2014	6/14/2014	Standard C	BH-11710	Brosina Hoffmar	Consumer	TEC-PH-10	Technology	Phones	Mitel 5320 IP F	907.152	6	0.2	90.7152
9	CA-2014-1	6/9/2014	6/14/2014	Standard C	BH-11710	Brosina Hoffmar	Consumer	OFF-BI-10	Office Sup	Binders	DXL Angle-Vi	18.504	3	0.2	5.7825
10	CA-2014-1	6/9/2014	6/14/2014	Standard C	BH-11710	Brosina Hoffmar	Consumer	OFF-AP-10	Office Sup	Appliances	Belkin F5C206	114.9	5	0	34.47

將 csv 資料切分

Customers

F	G	H
Customer ID	Customer Name	Segment
CG-12520	Claire Gute	Consumer
CG-12520	Claire Gute	Consumer
DV-13045	Darrin Van Huff	Corporate
SO-20335	Sean O'Donnell	Consumer
SO-20335	Sean O'Donnell	Consumer
BH-11710	Brosina Hoffmar	Consumer
BH-11710	Brosina Hoffmar	Consumer
BH-11710	Brosina Hoffmar	Consumer
BH-11710	Brosina Hoffmar	Consumer

Orders

B	C	D	E
Order ID	Order Date	Ship Date	Ship Mode
CA-2016-1	11/8/2016	11/11/2016	Second Cla
CA-2016-1	11/8/2016	11/11/2016	Second Cla
CA-2016-1	6/12/2016	6/16/2016	Second Cla
US-2015-1	10/11/2015	10/18/2015	Standard C
US-2015-1	10/11/2015	10/18/2015	Standard C
CA-2014-1	6/9/2014	6/14/2014	Standard C
CA-2014-1	6/9/2014	6/14/2014	Standard C
CA-2014-1	6/9/2014	6/14/2014	Standard C
CA-2014-1	6/9/2014	6/14/2014	Standard C

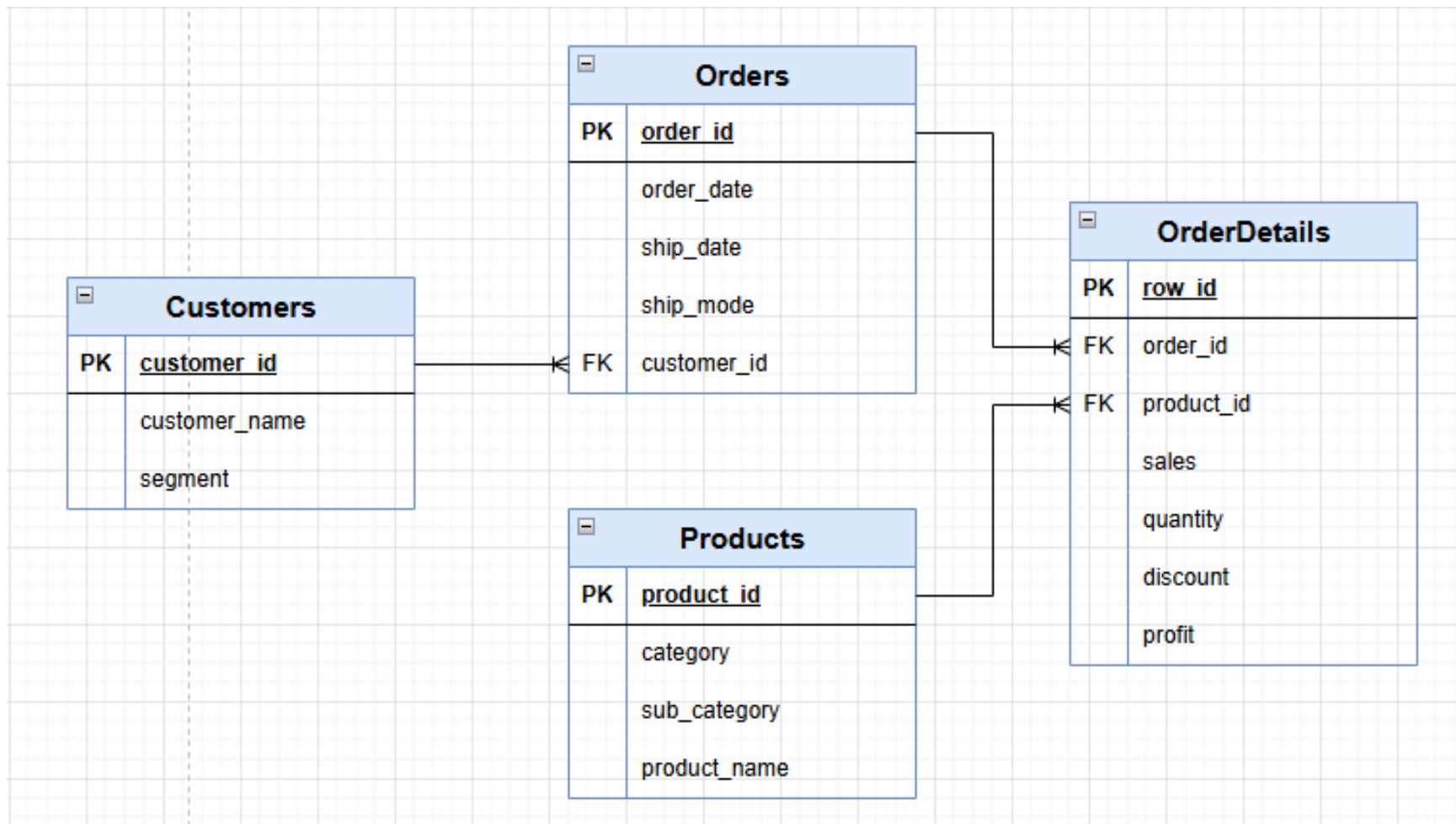
OrderDetails

M	N	O	P
Sales	Quantity	Discount	Profit
261.96	2	0	41.9136
731.94	3	0	219.582
14.62	2	0	6.8714
957.5775	5	0.45	-383.031
22.368	2	0.2	2.5164
48.86	7	0	14.1694
7.28	4	0	1.9656
907.152	6	0.2	90.7152
18.504	3	0.2	5.7825

Products

I	J	K	L
Product ID	Category	Sub-Category	Product Name
FUR-BO-10	Furniture	Bookcases	Bush Somerset
FUR-CH-10	Furniture	Chairs	Hon Deluxe Fa
OFF-LA-10	Office Sup	Labels	Self-Adhesive
FUR-TA-10	Furniture	Tables	Bretford CR450
OFF-ST-10	Office Sup	Storage	Eldon Fold 'N
FUR-FU-10	Furniture	Furnishings	Eldon Expressi
OFF-AR-10	Office Sup	Art	Newell 322
TEC-PH-10	Technology	Phones	Mitel 5320 IP F
OFF-BI-10	Office Sup	Binders	DXL Angle-Vi

將 csv 資料切分



建立資料表

```
sql = """Customers
CREATE TABLE IF NOT EXISTS Customers (
    customer_id VARCHAR(20) PRIMARY KEY,
    customer_name VARCHAR(100),
    segment VARCHAR(50)
);
"""
```

```
sql = """Orders
CREATE TABLE IF NOT EXISTS Orders (
    order_id VARCHAR(20) PRIMARY KEY,
    order_date DATE,
    ship_date DATE,
    ship_mode VARCHAR(50),
    customer_id VARCHAR(20),
    FOREIGN KEY (customer_id) REFERENCES Customers(customer_id)
);
"""
```

建立資料表

```
sql = """Products
CREATE TABLE IF NOT EXISTS Products (
    product_id VARCHAR(20) PRIMARY KEY,
    category VARCHAR(50),
    sub_category VARCHAR(50),
    product_name VARCHAR(255)
);
"""
```

```
sql = """
CREATE TABLE IF NOT EXISTS OrderDetails (
    row_id INT PRIMARY KEY,
    order_id VARCHAR(20),
    product_id VARCHAR(20),
    sales DECIMAL(10,2),
    quantity INT,
    discount DECIMAL(4,2),
    profit DECIMAL(10,2),
    FOREIGN KEY (order_id) REFERENCES Orders(order_id),
    FOREIGN KEY (product_id) REFERENCES Products(product_id)
);
"""OrderDetails
```

讀取 csv 資料 – 提取特定欄位

```
import pandas as pd

# 讀取 CSV 檔案
df = pd.read_csv("Sample-Superstore.csv", encoding='latin1')
# 提取客戶欄位的資料
customers = df[['Customer ID', 'Customer Name', 'Segment']]
customers
```

	Customer ID	Customer Name	Segment
0	CG-12520	Claire Gute	Consumer
1	CG-12520	Claire Gute	Consumer
2	DV-13045	Darrin Van Huff	Corporate
3	SO-20335	Sean O'Donnell	Consumer
4	SO-20335	Sean O'Donnell	Consumer
...	...	...	...
9989	TB-21400	Tom Boeckenhauer	Consumer
9990	DB-13060	Dave Brooks	Consumer
9991	DB-13060	Dave Brooks	Consumer
9992	DB-13060	Dave Brooks	Consumer
9993	CC-12220	Chris Cortes	Consumer
9994 rows × 3 columns			

讀取 csv 資料 – 去除重複數據

```
import pandas as pd

# 讀取 CSV 檔案
df = pd.read_csv("Sample-Superstore.csv", encoding='latin1')

# 提取客戶欄位的資料
customers = df[['Customer ID', 'Customer Name', 'Segment']]

# 去除重複的客戶資料
customers = customers.drop_duplicates()

customers
```

	Customer ID	Customer Name	Segment
0	CG-12520	Claire Gute	Consumer
2	DV-13045	Darrin Van Huff	Corporate
3	SO-20335	Sean O'Donnell	Consumer
5	BH-11710	Brosina Hoffman	Consumer
12	AA-10480	Andrew Allen	Consumer
...	...	...	...
8666	CJ-11875	Carl Jackson	Corporate
9209	RS-19870	Roy Skaria	Home Office
9399	SC-20845	Sung Chung	Consumer
9441	RE-19405	Ricardo Emerson	Consumer
9558	SM-20905	Susan MacKendrick	Consumer

793 rows × 3 columns

讀取 csv 資料 – 寫入資料庫

```
# 取得 df 中的客戶資料，並去除重複的資料
```

```
customers = df[['Customer ID', 'Customer Name', 'Segment']].drop_duplicates()
```

```
# 將客戶資料轉換為 list
```

```
customers_value_list = customers.values.tolist()
```

```
# 將資料寫入資料庫
```

```
with connection.cursor() as cursor:
```

```
    sql = """
```

```
    INSERT INTO Customers (customer_id, customer_name, segment) VALUES (%s, %s, %s)
```

```
    """
```

```
    cursor.executemany(sql, customers_value_list)
```

```
    connection.commit()
```

讀取 csv 資料 – 處理日期格式資料

```
# 取得 df 中的訂單資料
```

```
orders = df[['Order ID', 'Order Date', 'Ship Date', 'Ship Mode', 'Customer ID']].drop_duplicates()  
orders
```

	Order ID	Order Date	Ship Date	Ship Mode	Customer ID
0	CA-2016-152156	11/8/2016	11/11/2016	Second Class	CG-12520
2	CA-2016-138688	6/12/2016	6/16/2016	Second Class	DV-13045
3	US-2015-108966	10/11/2015	10/18/2015	Standard Class	SO-20335
5	CA-2014-115812	6/9/2014	6/14/2014	Standard Class	BH-11710
12	CA-2017-114412	4/15/2017	4/20/2017	Standard Class	AA-10480
...	...	...	...	...	...
9986	CA-2016-125794	9/29/2016	10/3/2016	Standard Class	ML-17410
9987	CA-2017-163629	11/17/2017	11/21/2017	Standard Class	RA-19885
9989	CA-2014-110422	1/21/2014	1/23/2014	Second Class	TB-21400
9990	CA-2017-121258	2/26/2017	3/3/2017	Standard Class	DB-13060
9993	CA-2017-119914	5/4/2017	5/9/2017	Second Class	CC-12220

5009 rows × 5 columns

讀取 csv 資料 – 處理日期格式資料

```
# 取得 df 中的訂單資料
```

```
orders = df[['Order ID', 'Order Date', 'Ship Date', 'Ship Mode', 'Customer ID']].drop_duplicates()
```

```
# 將日期格式轉換為 datetime.date
```

```
orders['Order Date'] = pd.to_datetime(orders['Order Date'], format='%m/%d/%Y').dt.date
```

```
orders['Ship Date'] = pd.to_datetime(orders['Ship Date'], format='%m/%d/%Y').dt.date
```

```
orders
```

	Order ID	Order Date	Ship Date	Ship Mode	Customer ID
0	CA-2016-152156	2016-11-08	2016-11-11	Second Class	CG-12520
2	CA-2016-138688	2016-06-12	2016-06-16	Second Class	DV-13045
3	US-2015-108966	2015-10-11	2015-10-18	Standard Class	SO-20335
5	CA-2014-115812	2014-06-09	2014-06-14	Standard Class	BH-11710
12	CA-2017-114412	2017-04-15	2017-04-20	Standard Class	AA-10480
...	...	...	...	...	...
9986	CA-2016-125794	2016-09-29	2016-10-03	Standard Class	ML-17410
9987	CA-2017-163629	2017-11-17	2017-11-21	Standard Class	RA-19885
9989	CA-2014-110422	2014-01-21	2014-01-23	Second Class	TB-21400
9990	CA-2017-121258	2017-02-26	2017-03-03	Standard Class	DB-13060
9993	CA-2017-119914	2017-05-04	2017-05-09	Second Class	CC-12220
5009 rows × 5 columns					